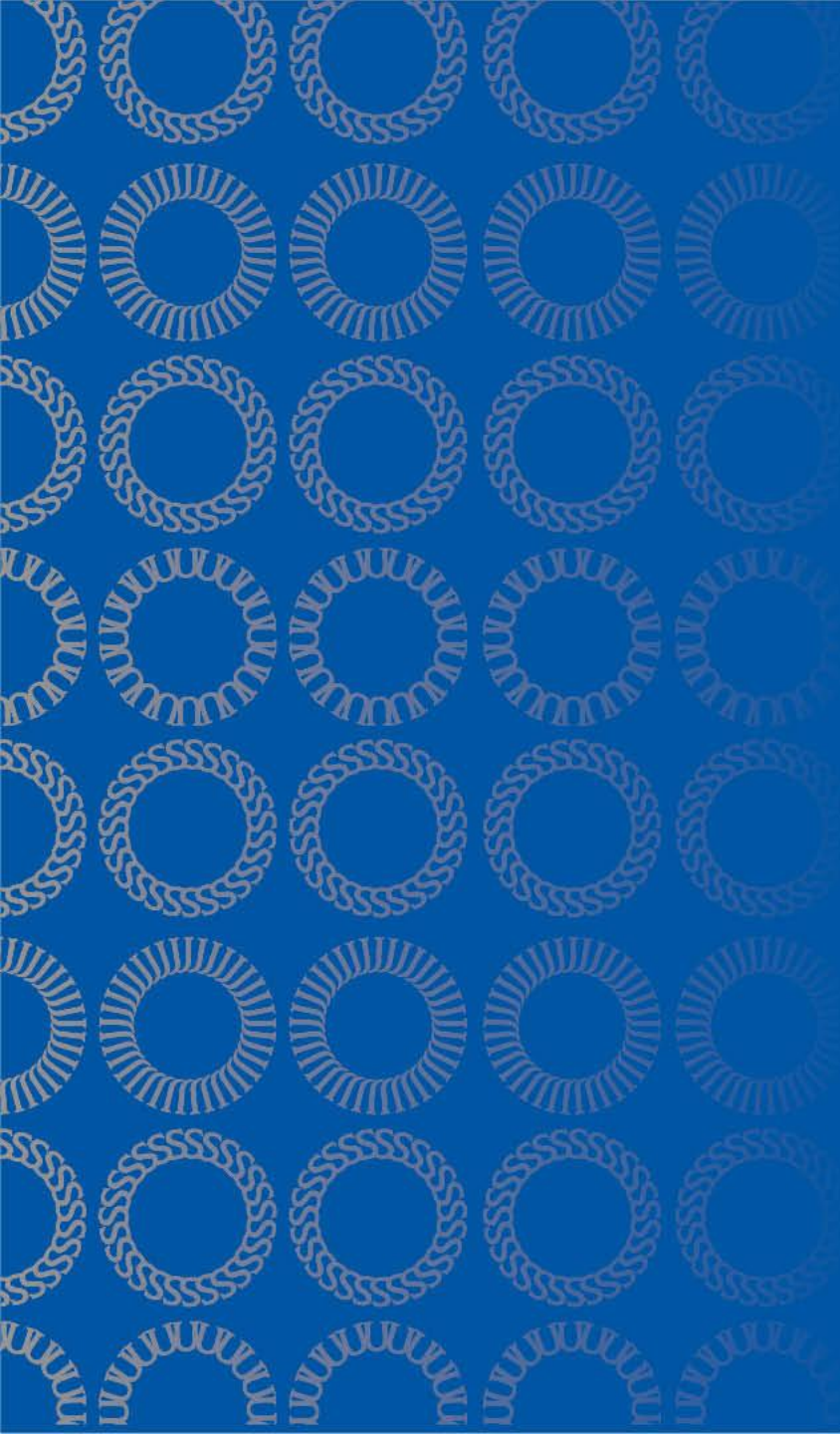




DATA 255

Deep Learning and Computer Vision in Applied Data Intelligence

Sept 23, 2026

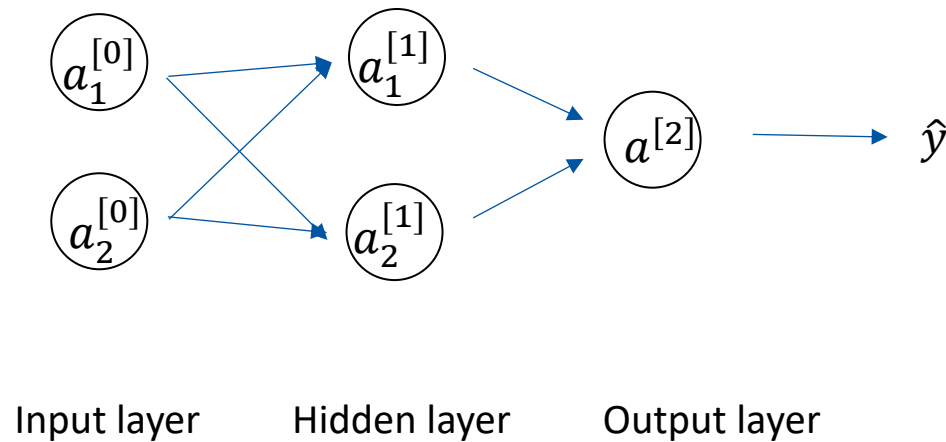
Taehee Jeong, Ph.D.

San Jose State University

Previous Topics

- Backward propagation for binary classification
- Backward propagation for multiclass classification

Forward pass for case2



Parameters: $W^{[1]}, W^{[2]}$

Cost function: $L(W^{[1]}, W^{[2]})$

$$J = \frac{1}{m} \sum_{i=1}^m L(a^{[2]}, y)$$

Forward propagation

1st layer

$$Z_1^{[1]} = W_{10}^{[1]} + W_{11}^{[1]} a_1^{[0]} + W_{12}^{[1]} a_2^{[0]}$$

$$Z_2^{[1]} = W_{20}^{[1]} + W_{21}^{[1]} a_1^{[0]} + W_{22}^{[1]} a_2^{[0]}$$

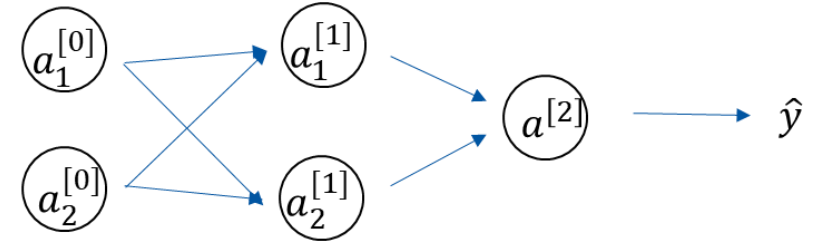
$$a_1^{[1]} = \sigma(Z_1^{[1]})$$

$$a_2^{[1]} = \sigma(Z_2^{[1]})$$

2nd layer

$$Z^{[2]} = W_0^{[2]} + W_1^{[2]} a_1^{[1]} + W_2^{[2]} a_2^{[1]}$$

$$a^{[2]} = \sigma(Z^{[2]})$$



Input layer

Hidden layer

Output layer

Derivative of $Z^{[2]}$

backpropagation for binary entropy loss

$$L(a^{[2]}, y) = -[y * \log(a^{[2]}) + (1 - y) * \log(1 - a^{[2]})]$$

$$dZ^{[2]} = \frac{\partial L}{\partial a^{[2]}} \frac{\partial a^{[2]}}{\partial Z^{[2]}}$$

$$dZ^{[2]} = (a^{[2]} - y)$$

**Element wise multiplication*

1 st layer	$Z_1^{[1]} = W_{10}^{[1]} + W_{11}^{[1]} a_1^{[0]} + W_{12}^{[1]} a_2^{[0]}$
	$Z_2^{[1]} = W_{20}^{[1]} + W_{21}^{[1]} a_1^{[0]} + W_{22}^{[1]} a_2^{[0]}$
	$a_1^{[1]} = \sigma(Z_1^{[1]})$
	$a_2^{[1]} = \sigma(Z_2^{[1]})$
2 nd layer	$Z^{[2]} = W_0^{[2]} + W_1^{[2]} a_1^{[1]} + W_2^{[2]} a_2^{[1]}$
	$a^{[2]} = \sigma(Z^{[2]})$

Derivative of $W_0^{[2]}$

backpropagation for binary entropy loss

$$L(a^{[2]}, y) = -[y * \log(a^{[2]}) + (1 - y) * \log(1 - a^{[2]})]$$

$$\frac{\partial L(a, y)}{\partial W_0^{[2]}} = dW_0^{[2]}$$

$$\begin{aligned} dW_0^{[2]} &= \frac{\partial L}{\partial a^{[2]}} \frac{\partial a^{[2]}}{\partial Z^{[2]}} \frac{\partial Z^{[2]}}{\partial W_0^{[2]}} \\ &= dZ^{[2]} \frac{\partial Z^{[2]}}{\partial W_0^{[2]}} \end{aligned}$$

$$dW_0^{[2]} = dZ^{[2]} \cdot 1$$

**Element wise multiplication*

1 st layer	$Z_1^{[1]} = W_{10}^{[1]} + W_{11}^{[1]} a_1^{[0]} + W_{12}^{[1]} a_2^{[0]}$
	$Z_2^{[1]} = W_{20}^{[1]} + W_{21}^{[1]} a_1^{[0]} + W_{22}^{[1]} a_2^{[0]}$
	$a_1^{[1]} = \sigma(Z_1^{[1]})$
	$a_2^{[1]} = \sigma(Z_2^{[1]})$
2 nd layer	$Z^{[2]} = W_0^{[2]} + W_1^{[2]} a_1^{[1]} + W_2^{[2]} a_2^{[1]}$
	$a^{[2]} = \sigma(Z^{[2]})$

Derivative of $W_1^{[2]}$

backpropagation for binary classification

$$L(a^{[2]}, y) = -[y * \log(a^{[2]}) + (1 - y) * \log(1 - a^{[2]})]$$

$$\frac{\partial L(a^{[2]}, y)}{\partial W_1^{[2]}} = dW_1^{[2]}$$

$$\begin{aligned} dW_1^{[2]} &= \frac{\partial L}{\partial a^{[2]}} \frac{\partial a^{[2]}}{\partial Z^{[2]}} \frac{\partial Z^{[2]}}{\partial W_1^{[2]}} \\ &= dZ^{[2]} \frac{\partial Z^{[2]}}{\partial W_1^{[2]}} \end{aligned}$$

$$dW_1^{[2]} = dZ^{[2]} \cdot a_1^{[1]}$$

**Element wise multiplication*

1 st layer	$Z_1^{[1]} = W_{10}^{[1]} + W_{11}^{[1]} a_1^{[0]} + W_{12}^{[1]} a_2^{[0]}$
	$Z_2^{[1]} = W_{20}^{[1]} + W_{21}^{[1]} a_1^{[0]} + W_{22}^{[1]} a_2^{[0]}$
	$a_1^{[1]} = \sigma(Z_1^{[1]})$
	$a_2^{[1]} = \sigma(Z_2^{[1]})$
2 nd layer	$Z^{[2]} = W_0^{[2]} + W_1^{[2]} a_1^{[1]} + W_2^{[2]} a_2^{[1]}$
	$a^{[2]} = \sigma(Z^{[2]})$

Derivative of $W_2^{[2]}$

backpropagation for binary classification

$$L(a^{[2]}, y) = -[y * \log(a^{[2]}) + (1 - y) * \log(1 - a^{[2]})]$$

$$\frac{\partial L(a^{[2]}, y)}{\partial W_2^{[2]}} = dW_2^{[2]}$$

$$\begin{aligned} dW_2^{[2]} &= \frac{\partial L}{\partial a^{[2]}} \frac{\partial a^{[2]}}{\partial Z^{[2]}} \frac{\partial Z^{[2]}}{\partial W_2^{[2]}} \\ &= dZ^{[2]} \frac{\partial Z^{[2]}}{\partial W_2^{[2]}} \end{aligned}$$

$$dW_2^{[2]} = dZ^{[2]} \cdot a_2^{[1]}$$

**Element wise multiplication*

1 st layer	$Z_1^{[1]} = W_{10}^{[1]} + W_{11}^{[1]} a_1^{[0]} + W_{12}^{[1]} a_2^{[0]}$
	$Z_2^{[1]} = W_{20}^{[1]} + W_{21}^{[1]} a_1^{[0]} + W_{22}^{[1]} a_2^{[0]}$
	$a_1^{[1]} = \sigma(Z_1^{[1]})$
	$a_2^{[1]} = \sigma(Z_2^{[1]})$
2 nd layer	$Z^{[2]} = W_0^{[2]} + W_1^{[2]} a_1^{[1]} + W_2^{[2]} a_2^{[1]}$
	$a^{[2]} = \sigma(Z^{[2]})$

Derivative of Parameters in 2nd layer

backpropagation for binary entropy loss

$$L(a^{[2]}, y) = -[y * \log(a^{[2]}) + (1 - y) * \log(1 - a^{[2]})]$$

$$dZ^{[2]} = a^{[2]} - y$$

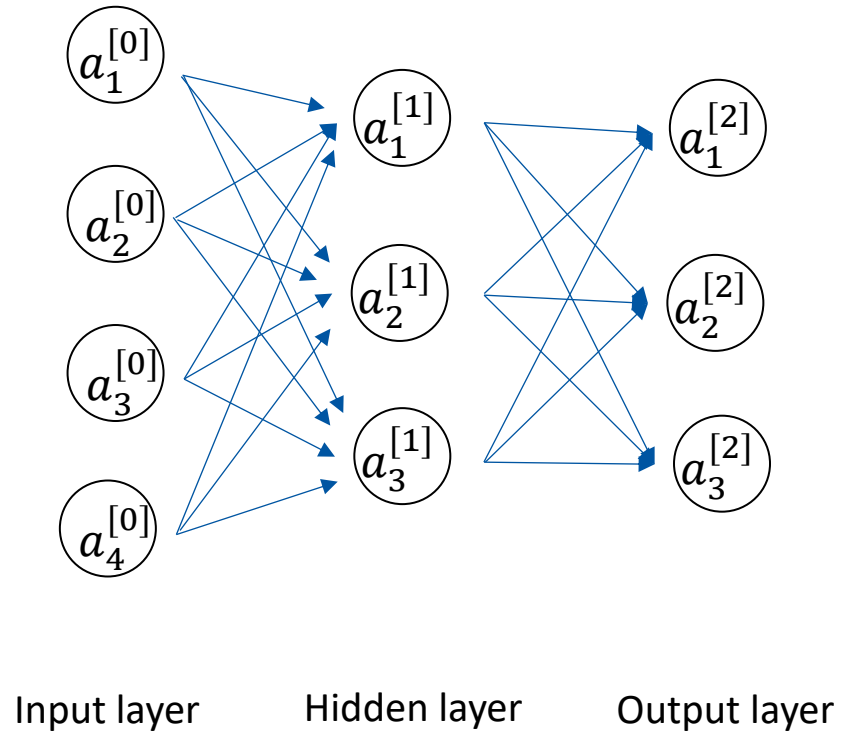
$$dW_0^{[2]} = dZ^{[2]} \cdot 1$$

$$dW_1^{[2]} = dZ^{[2]} \cdot a_1^{[1]}$$

$$dW_2^{[2]} = dZ^{[2]} \cdot a_2^{[1]}$$

1 st layer	$Z_1^{[1]} = W_{10}^{[1]} + W_{11}^{[1]} a_1^{[0]} + W_{12}^{[1]} a_2^{[0]}$
	$Z_2^{[1]} = W_{20}^{[1]} + W_{21}^{[1]} a_1^{[0]} + W_{22}^{[1]} a_2^{[0]}$
	$a_1^{[1]} = \sigma(Z_1^{[1]})$
	$a_2^{[1]} = \sigma(Z_2^{[1]})$
2 nd layer	$Z^{[2]} = W_0^{[2]} + W_1^{[2]} a_1^{[1]} + W_2^{[2]} a_2^{[1]}$
	$a^{[2]} = \sigma(Z^{[2]})$

Neural Network: 3 output neurons



Activation function of the last layer is *Softmax*.

Forward propagation

1st layer

$$Z_1^{[1]} = W_{10}^{[1]} + W_{11}^{[1]} a_1^{[0]} + W_{12}^{[1]} a_2^{[0]} + W_{13}^{[1]} a_3^{[0]} + W_{14}^{[1]} a_4^{[0]}$$

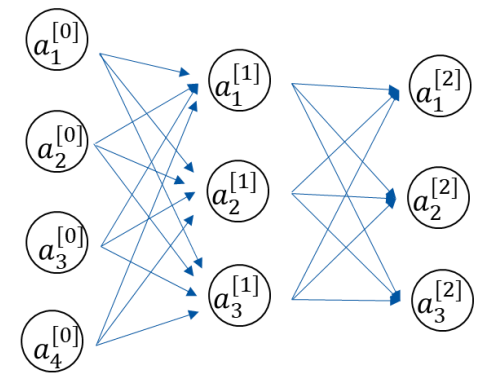
$$Z_2^{[1]} = W_{20}^{[1]} + W_{21}^{[1]} a_1^{[0]} + W_{22}^{[1]} a_2^{[0]} + W_{23}^{[1]} a_3^{[0]} + W_{24}^{[1]} a_4^{[0]}$$

$$Z_3^{[1]} = W_{30}^{[1]} + W_{31}^{[1]} a_1^{[0]} + W_{32}^{[1]} a_2^{[0]} + W_{33}^{[1]} a_3^{[0]} + W_{34}^{[1]} a_4^{[0]}$$

$$a_1^{[1]} = \sigma(Z_1^{[1]})$$

$$a_2^{[1]} = \sigma(Z_2^{[1]})$$

$$a_3^{[1]} = \sigma(Z_3^{[1]})$$



Input layer

Hidden layer

Output layer

Forward propagation

2nd layer

$$Z_1^{[2]} = W_{10}^{[2]} + W_{11}^{[2]} a_1^{[1]} + W_{12}^{[2]} a_2^{[1]} + W_{13}^{[2]} a_3^{[1]}$$

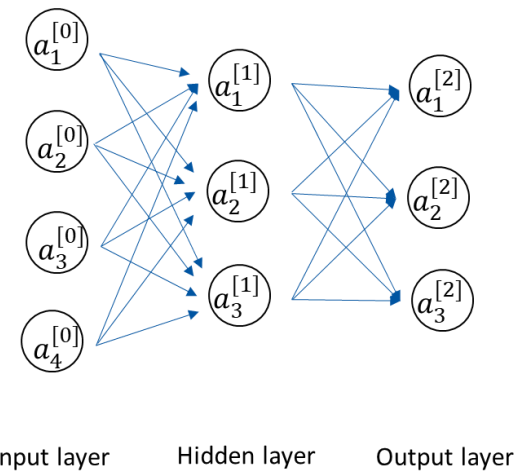
$$Z_2^{[2]} = W_{20}^{[2]} + W_{21}^{[2]} a_1^{[1]} + W_{22}^{[2]} a_2^{[1]} + W_{23}^{[2]} a_3^{[1]}$$

$$Z_3^{[2]} = W_{30}^{[2]} + W_{31}^{[2]} a_1^{[1]} + W_{32}^{[2]} a_2^{[1]} + W_{33}^{[2]} a_3^{[1]}$$

$$a_1^{[2]} = S(Z_1^{[2]})$$

$$a_2^{[2]} = S(Z_2^{[2]})$$

$$a_3^{[2]} = S(Z_3^{[2]})$$



Derivative of $a^{[2]}$

backpropagation for cross entropy loss

$$L(\hat{y}, y) = -[y_1 * \log \hat{y}_1 + y_2 * \log \hat{y}_2 + y_3 * \log \hat{y}_3]$$

$$L(\hat{y}, y) = -[y_1 * \log(a_1^{[2]}) + y_2 * \log(a_2^{[2]}) + y_3 * \log(a_3^{[2]})]$$

Activation function of output layer is softmax

$$a_1^{[2]} = \frac{\exp(Z_1^{[2]})}{\sum_{j=1}^K \exp(Z_j^{[2]})}$$

$$a_1^{[2]} = \frac{\exp(Z_1^{[2]})}{\exp(Z_1^{[2]}) + \exp(Z_2^{[2]}) + \exp(Z_3^{[2]})}$$

$$a_2^{[2]} = \frac{\exp(Z_2^{[2]})}{\exp(Z_1^{[2]}) + \exp(Z_2^{[2]}) + \exp(Z_3^{[2]})}$$

$$a_3^{[2]} = \frac{\exp(Z_3^{[2]})}{\exp(Z_1^{[2]}) + \exp(Z_2^{[2]}) + \exp(Z_3^{[2]})}$$

Derivative of *softmax*

$$S(Z_1) = \frac{\exp(Z_1)}{\exp(Z_1) + \exp(Z_2) + \exp(Z_3)}$$

$$\frac{\partial S(Z_j)}{\partial Z_i} = S(Z_i) \times (1 - S(Z_i)) \quad \text{if } i = j$$

$$\frac{\partial S(Z_j)}{\partial Z_i} = -S(Z_i) \times S(Z_j) \quad \text{if } i \neq j$$

Chain rule

$$h(x) = f(x) \cdot g(x)$$

$$\frac{d}{dx} h(x) = \left[\frac{d}{dx} f(x) \right] \cdot g(x) + f(x) \cdot \left[\frac{d}{dx} g(x) \right]$$

$$\frac{d}{dx} e^x = e^x$$

$$\frac{d}{dx} e^{-x} = -e^{-x}$$

Derivative of $Z^{[2]}$

$$L(a_i^{[2]}, y) = -[y_1 * \log(a_1^{[2]}) + y_2 * \log(a_2^{[2]}) + y_3 * \log(a_3^{[2]})]$$

$$L(a_i^{[2]}, y) = -[y_1 * \log(S(Z_1^{[2]})) + y_2 * \log(S(Z_2^{[2]})) + y_3 * \log(S(Z_3^{[2]}))]$$

$$dZ_1^{[2]} = \frac{\partial L}{\partial a_1^{[2]}} \frac{\partial a_1^{[2]}}{\partial Z_1^{[2]}}$$

$$dZ_2^{[2]} = \frac{\partial L}{\partial a_2^{[2]}} \frac{\partial a_2^{[2]}}{\partial Z_1^{[2]}}$$

$$dZ_3^{[2]} = \frac{\partial L}{\partial a_3^{[2]}} \frac{\partial a_3^{[2]}}{\partial Z_1^{[2]}}$$

$$a_1^{[2]} = S(Z_1^{[2]}) = \frac{\exp(Z_1^{[2]})}{\exp(Z_1^{[2]}) + \exp(Z_2^{[2]}) + \exp(Z_3^{[2]})}$$

$$\frac{\partial S(Z_j)}{\partial Z_i} = S(Z_i) \times (1 - S(Z_i)) \quad \text{if } i = j$$

$$\frac{\partial S(Z_j)}{\partial Z_i} = -S(Z_i) \times S(Z_j) \quad \text{if } i \neq j$$

Derivative of $dZ_1^{[2]}$

$$L(\hat{y}, y) = -[y_1 * \log(a_1^{[2]}) + y_2 * \log(a_2^{[2]}) + y_3 * \log(a_3^{[2]})]$$

$$L(\hat{y}, y) = -[y_1 * \log(S(Z_1^{[2]})) + y_2 * \log(S(Z_2^{[2]})) + y_3 * \log(S(Z_3^{[2]}))]$$

$$dZ_1^{[2]} = -\frac{y_1}{a_1^{[2]}} \cdot \frac{\partial S(Z_1^{[2]})}{\partial Z_1^{[2]}} - \frac{y_2}{a_2^{[2]}} \cdot \frac{\partial S(Z_2^{[2]})}{\partial Z_1^{[2]}} - \frac{y_3}{a_3^{[2]}} \cdot \frac{\partial S(Z_3^{[2]})}{\partial Z_1^{[2]}}$$

$$dZ_1^{[2]} = -y_1 \cdot (1 - a_1^{[2]}) + y_2 \cdot a_1^{[2]} + y_3 \cdot a_1^{[2]}$$

$$dZ_1^{[2]} = -y_1 + y_1 \cdot a_1^{[2]} + y_2 \cdot a_1^{[2]} + y_3 \cdot a_1^{[2]}$$

$$dZ_1^{[2]} = -y_1 + a_1^{[2]}$$

$$dZ_1^{[2]} = a_1^{[2]} - y_1$$

$$a_1^{[2]} = S(Z_1^{[2]}) = \frac{\exp(Z_1^{[2]})}{\exp(Z_1^{[2]}) + \exp(Z_2^{[2]}) + \exp(Z_3^{[2]})}$$

$$\frac{\partial S(Z_j)}{\partial Z_i} = S(Z_i) \times (1 - S(Z_i))$$

$$\frac{\partial S(Z_j)}{\partial Z_i} = -S(Z_i) \times S(Z_j)$$

Derivative of $dZ_2^{[2]}$

$$L(\hat{y}, y) = -[y_1 * \log(a_1^{[2]}) + y_2 * \log(a_2^{[2]}) + y_3 * \log(a_3^{[2]})]$$

$$L(\hat{y}, y) = -[y_1 * \log(S(Z_1^{[2]})) + y_2 * \log(S(Z_2^{[2]})) + y_3 * \log(S(Z_3^{[2]}))]$$

$$dZ_2^{[2]} = -\frac{y_1}{a_1^{[2]}} \cdot \frac{\partial S(Z_1^{[2]})}{\partial Z_2^{[2]}} - \frac{y_2}{a_2^{[2]}} \cdot \frac{\partial S(Z_2^{[2]})}{\partial Z_2^{[2]}} - \frac{y_3}{a_3^{[2]}} \cdot \frac{\partial S(Z_3^{[2]})}{\partial Z_2^{[2]}}$$

$$dZ_2^{[2]} = y_1 \cdot a_2^{[2]} - y_2 \cdot (1 - a_2^{[2]}) + y_3 \cdot a_2^{[2]}$$

$$dZ_2^{[2]} = -y_2 + y_1 \cdot a_2^{[2]} + y_2 \cdot a_2^{[2]} + y_3 \cdot a_2^{[2]}$$

$$dZ_2^{[2]} = -y_2 + a_2^{[2]}$$

$$dZ_2^{[2]} = a_2^{[2]} - y_2$$

$$a_2^{[2]} = S(Z_2^{[2]}) = \frac{\exp(Z_2^{[2]})}{\exp(Z_1^{[2]}) + \exp(Z_2^{[2]}) + \exp(Z_3^{[2]})}$$

$$\frac{\partial S(Z_j)}{\partial Z_i} = S(Z_i) \times (1 - S(Z_i))$$

$$\frac{\partial S(Z_j)}{\partial Z_i} = -S(Z_i) \times S(Z_j)$$

Derivative of $dZ_3^{[2]}$

$$L(\hat{y}, y) = -[y_1 * \log(a_1^{[2]}) + y_2 * \log(a_2^{[2]}) + y_3 * \log(a_3^{[2]})]$$

$$L(\hat{y}, y) = -[y_1 * \log(S(Z_1^{[2]})) + y_2 * \log(S(Z_2^{[2]})) + y_3 * \log(S(Z_3^{[2]}))]$$

$$dZ_3^{[2]} = -\frac{y_1}{a_1^{[2]}} \cdot \frac{\partial S(Z_1^{[2]})}{\partial Z_3^{[2]}} - \frac{y_2}{a_2^{[2]}} \cdot \frac{\partial S(Z_2^{[2]})}{\partial Z_3^{[2]}} - \frac{y_3}{a_3^{[2]}} \cdot \frac{\partial S(Z_3^{[2]})}{\partial Z_3^{[2]}}$$

$$dZ_3^{[2]} = y_1 \cdot a_3^{[2]} + y_2 \cdot a_3^{[2]} - y_3 \cdot (1 - a_3^{[2]})$$

$$dZ_3^{[2]} = -y_3 + y_1 \cdot a_3^{[2]} + y_2 \cdot a_3^{[2]} + y_3 \cdot a_3^{[2]}$$

$$dZ_3^{[2]} = -y_3 + a_3^{[2]}$$

$$dZ_3^{[2]} = a_3^{[2]} - y_3$$

$$a_3^{[2]} = S(Z_3^{[2]}) = \frac{\exp(Z_3^{[2]})}{\exp(Z_1^{[2]}) + \exp(Z_2^{[2]}) + \exp(Z_3^{[2]})}$$

$$\frac{\partial S(Z_j)}{\partial Z_i} = S(Z_i) \times (1 - S(Z_i))$$

$$\frac{\partial S(Z_j)}{\partial Z_i} = -S(Z_i) \times S(Z_j)$$

2nd layer

$$Z_1^{[2]} = W_{10}^{[2]} + W_{11}^{[2]} a_1^{[1]} + W_{12}^{[2]} a_2^{[1]} + W_{13}^{[2]} a_3^{[1]}$$

$$Z_2^{[2]} = W_{20}^{[2]} + W_{21}^{[2]} a_1^{[1]} + W_{22}^{[2]} a_2^{[1]} + W_{23}^{[2]} a_3^{[1]}$$

$$Z_3^{[2]} = W_{30}^{[2]} + W_{31}^{[2]} a_1^{[1]} + W_{32}^{[2]} a_2^{[1]} + W_{33}^{[2]} a_3^{[1]}$$

$$a_1^{[2]} = s(Z_1^{[2]})$$

$$a_2^{[2]} = s(Z_2^{[2]})$$

$$a_3^{[2]} = s(Z_3^{[2]})$$

Derivative of $W_{10}^{[2]}$

$$L(a_i^{[2]}, y) = -[y_1 * \log(a_1^{[2]}) + y_2 * \log(a_2^{[2]}) + y_3 * \log(a_3^{[2]})]$$

$$\frac{\partial L(a, y)}{\partial W_{10}^{[2]}} = dW_{10}^{[2]}$$

$$\begin{aligned} dW_{10}^{[2]} &= \frac{\partial L}{\partial a_1^{[2]}} \frac{\partial a_1^{[2]}}{\partial Z_1^{[2]}} \frac{\partial Z_1^{[2]}}{\partial W_{10}^{[2]}} \\ &= dZ_1^{[2]} \frac{\partial Z_1^{[2]}}{\partial W_{10}^{[2]}} \end{aligned}$$

$$dW_{10}^{[2]} = dZ_1^{[2]} \cdot 1$$

2nd layer

$$Z_1^{[2]} = W_{10}^{[2]} + W_{11}^{[2]} a_1^{[1]} + W_{12}^{[2]} a_2^{[1]} + W_{13}^{[2]} a_3^{[1]}$$

$$Z_2^{[2]} = W_{20}^{[2]} + W_{21}^{[2]} a_1^{[1]} + W_{22}^{[2]} a_2^{[1]} + W_{23}^{[2]} a_3^{[1]}$$

$$Z_3^{[2]} = W_{30}^{[2]} + W_{31}^{[2]} a_1^{[1]} + W_{32}^{[2]} a_2^{[1]} + W_{33}^{[2]} a_3^{[1]}$$

$$a_1^{[2]} = S(Z_1^{[2]})$$

$$a_2^{[2]} = S(Z_2^{[2]})$$

$$a_3^{[2]} = S(Z_3^{[2]})$$

Derivative of $W_{11}^{[2]}$

$$L(a_i^{[2]}, y) = -[y_1 * \log(a_1^{[2]}) + y_2 * \log(a_2^{[2]}) + y_3 * \log(a_3^{[2]})]$$

$$\frac{\partial L(a, y)}{\partial W_{11}^{[2]}} = dW_{11}^{[2]}$$

$$\begin{aligned} dW_{11}^{[2]} &= \frac{\partial L}{\partial a_1^{[2]}} \frac{\partial a_1^{[2]}}{\partial Z_1^{[2]}} \frac{\partial Z_1^{[2]}}{\partial W_{11}^{[2]}} \\ &= dZ_1^{[2]} \frac{\partial Z_1^{[2]}}{\partial W_{11}^{[2]}} \end{aligned}$$

$$dW_{11}^{[2]} = dZ_1^{[2]} \cdot a_1^{[1]}$$

2nd layer

$$Z_1^{[2]} = W_{10}^{[2]} + W_{11}^{[2]} a_1^{[1]} + W_{12}^{[2]} a_2^{[1]} + W_{13}^{[2]} a_3^{[1]}$$

$$Z_2^{[2]} = W_{20}^{[2]} + W_{21}^{[2]} a_1^{[1]} + W_{22}^{[2]} a_2^{[1]} + W_{23}^{[2]} a_3^{[1]}$$

$$Z_3^{[2]} = W_{30}^{[2]} + W_{31}^{[2]} a_1^{[1]} + W_{32}^{[2]} a_2^{[1]} + W_{33}^{[2]} a_3^{[1]}$$

$$a_1^{[2]} = S(Z_1^{[2]})$$

$$a_2^{[2]} = S(Z_2^{[2]})$$

$$a_3^{[2]} = S(Z_3^{[2]})$$

Derivative of $W_{12}^{[2]}$

$$L(a_i^{[2]}, y) = -[y_1 * \log(a_1^{[2]}) + y_2 * \log(a_2^{[2]}) + y_3 * \log(a_3^{[2]})]$$

$$\frac{\partial L(a, y)}{\partial W_{12}^{[2]}} = dW_{12}^{[2]}$$

$$\begin{aligned} dW_{12}^{[2]} &= \frac{\partial L}{\partial a_1^{[2]}} \frac{\partial a_1^{[2]}}{\partial Z_1^{[2]}} \frac{\partial Z_1^{[2]}}{\partial W_{12}^{[2]}} \\ &= dZ_1^{[2]} \frac{\partial Z_1^{[2]}}{\partial W_{12}^{[2]}} \end{aligned}$$

$$dW_{12}^{[2]} = dZ_1^{[2]} \cdot a_2^{[1]}$$

Derivative of $W_{13}^{[2]}$

$$L(a_i^{[2]}, y) = -[y_1 * \log(a_1^{[2]}) + y_2 * \log(a_2^{[2]}) + y_3 * \log(a_3^{[2]})]$$

$$\frac{\partial L(a, y)}{\partial W_{13}^{[2]}} = dW_{13}^{[2]}$$

$$\begin{aligned} dW_{13}^{[2]} &= \frac{\partial L}{\partial a_1^{[2]}} \frac{\partial a_1^{[2]}}{\partial Z_1^{[2]}} \frac{\partial Z_1^{[2]}}{\partial W_{13}^{[2]}} \\ &= dZ_1^{[2]} \frac{\partial Z_1^{[2]}}{\partial W_{13}^{[2]}} \end{aligned}$$

$$dW_{13}^{[2]} = dZ_1^{[2]} \cdot a_3^{[1]}$$

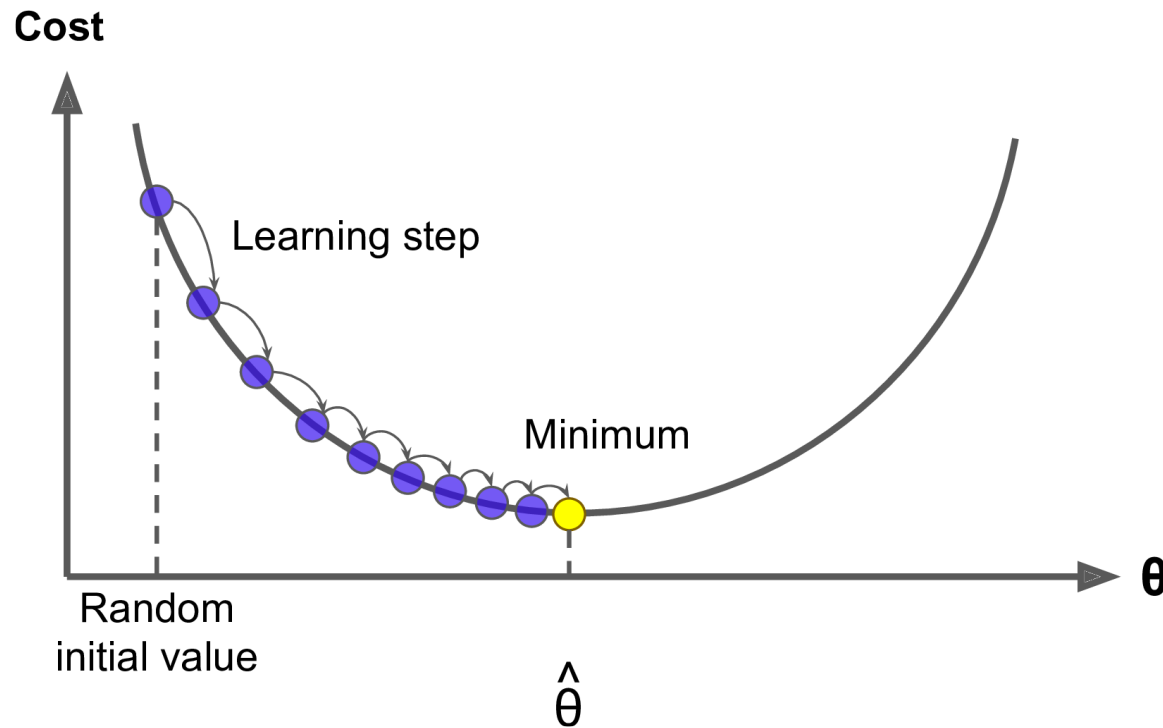
Assignment5

- Calculate backpropagation for cross entropy loss
- Train the neural network for multiclass classification

Today's Topics(1)

- Learning rate decay
- Optimizer

Gradient Descent



$$W_j^t := W_j^{t-1} - \alpha \frac{\partial J}{\partial W}$$

Batch & Stochastic gradient descent

**Batch
Gradient
descent**

$$\frac{\partial J}{\partial W} = -\frac{1}{m} \sum_{i=1}^m [(\hat{y}^i - y^i)x_j^i] \quad (\text{Sum over data points})$$

**Stochastic
Gradient
descent**

$$\frac{\partial J}{\partial W} = -(\hat{y}^i - y^i)x_j^i \quad (\text{Each time, pick different data point})$$

Mini-batch Gradient Descent

- Batch Gradient Descent

Computes the gradients based on full training set

Ex) Offline learning

- Stochastic Gradient Descent

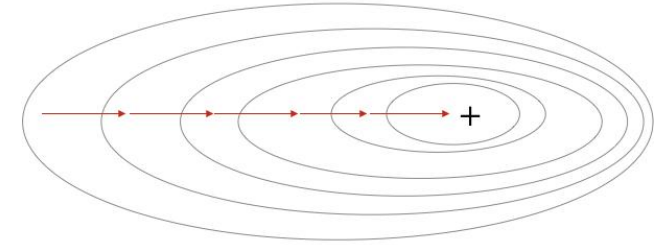
Computes just one instance

Ex) Online learning

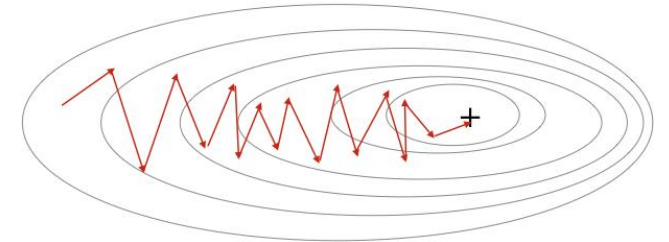
- Mini-batch Gradient Descent

Computes the gradients on small random sets of instances

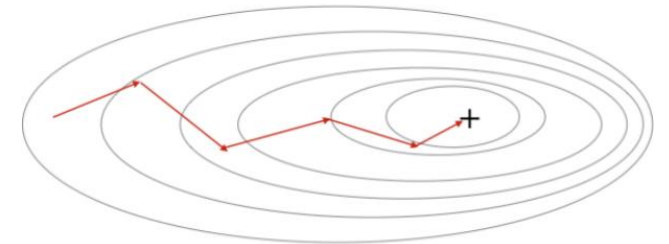
Gradient Descent



Stochastic Gradient Descent



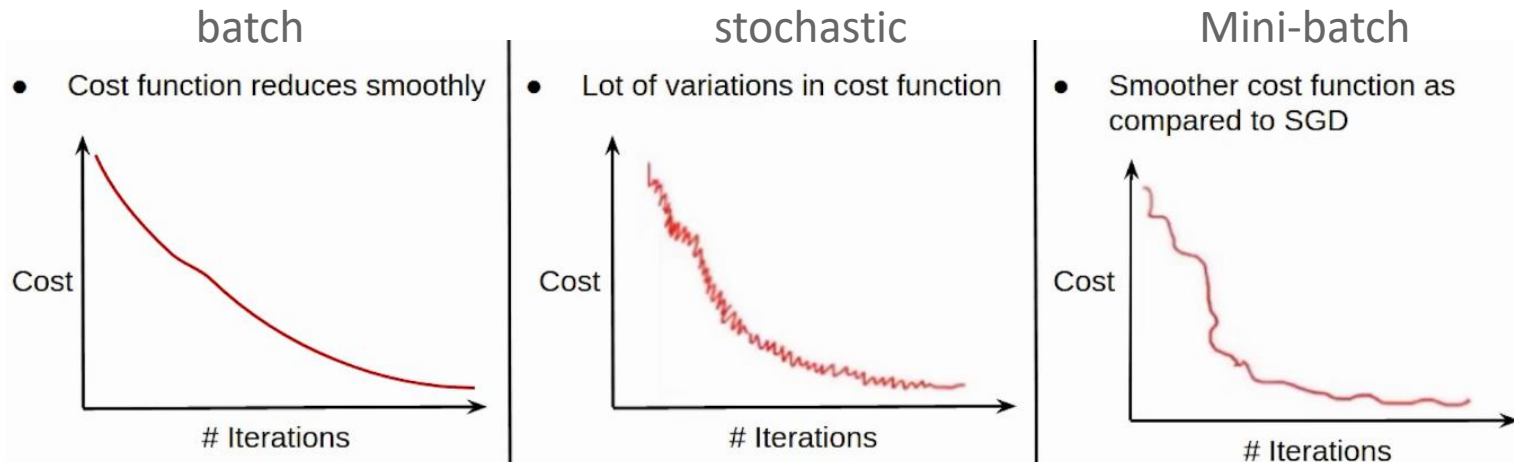
Mini-Batch Gradient Descent



1 epoch : 1 pass through 1 batch, whole data set,
1 iteration : 1 pass through 1 mini-batch

Batch & Stochastic gradient descent

- "batch gradient descent" calculates the gradient using the entire training dataset for each parameter update
- "stochastic gradient descent" updates parameters based on the gradient calculated from just one randomly selected data point at a time, making it significantly faster for large datasets but potentially less stable due to the noisy updates



When Learning rate is too high

If learning rate is large, parameters decay too aggressively and the parameters are unable to reach the local minimum.

$$W_j^t := W_j^{t-1} - \alpha \frac{\partial J}{\partial W}$$

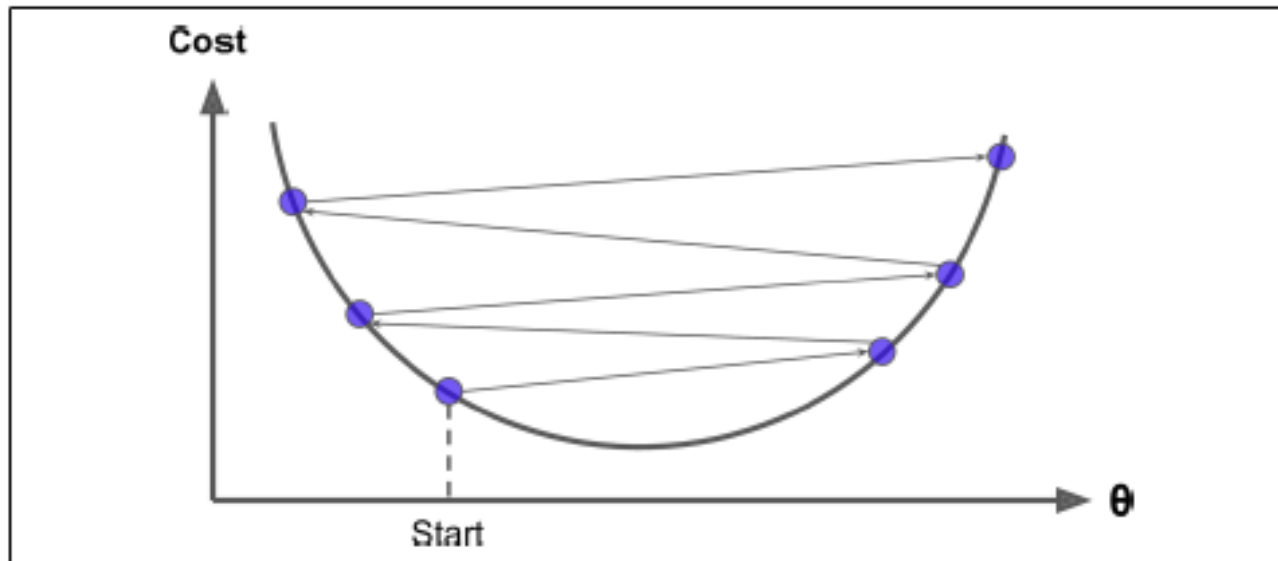
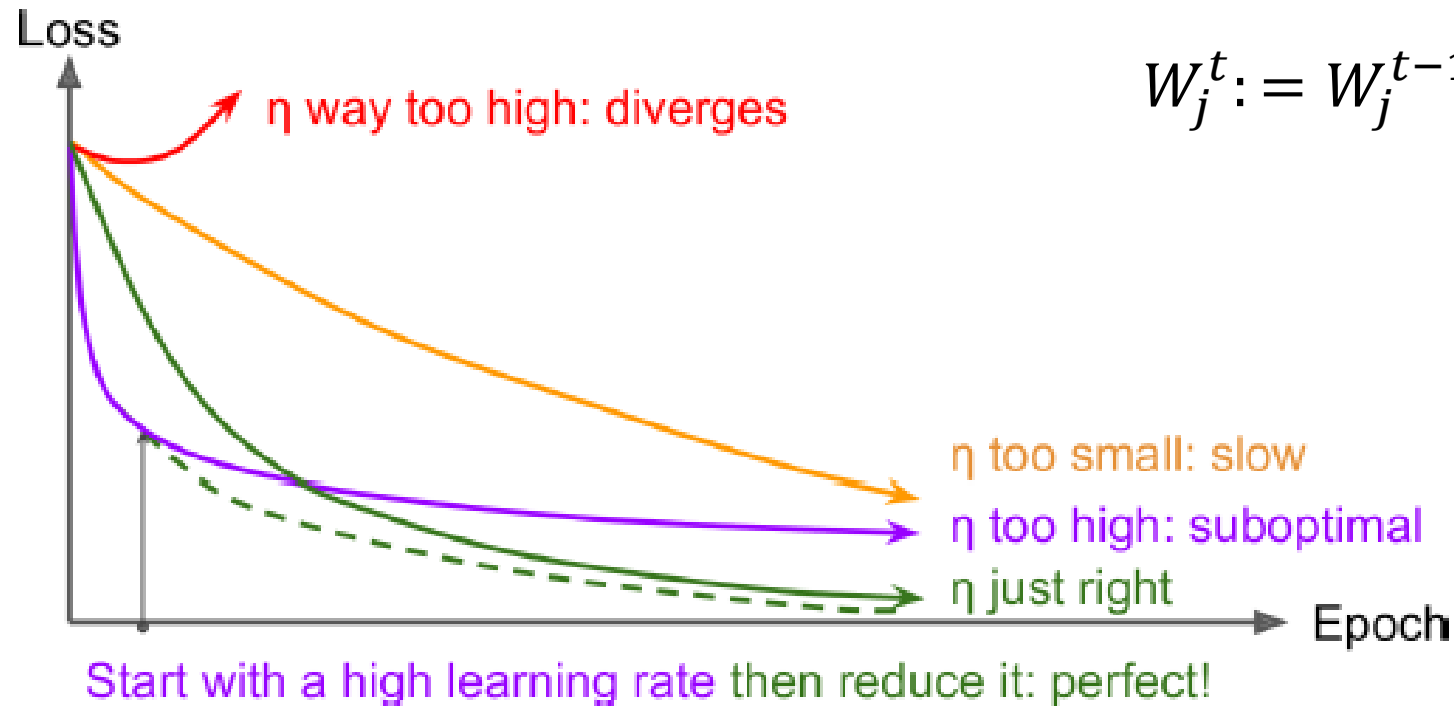


Figure 4-5. Learning rate too large

Loss vs. Learning rate (or step size)

1 epoch : 1 pass through data set, iteration number



$$W_j^t := W_j^{t-1} - \alpha \frac{\partial J}{\partial W}$$

Learning rate decay

Slowing decreasing α

- $1/t$ decay
- Exponentially decay
- Step decay: discrete staircase
- Discrete staircase

1/t decay

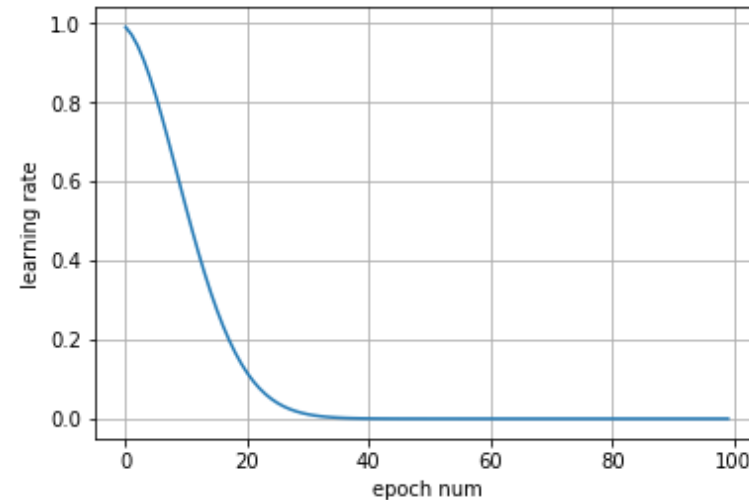
$$\alpha = \frac{\alpha_0}{1 + \text{decay_rate} \times \text{epoch_num}} = \frac{\alpha_0}{1 + k \cdot t}$$

$$\alpha_0 = 1$$

decay_rate(k) = 0.1



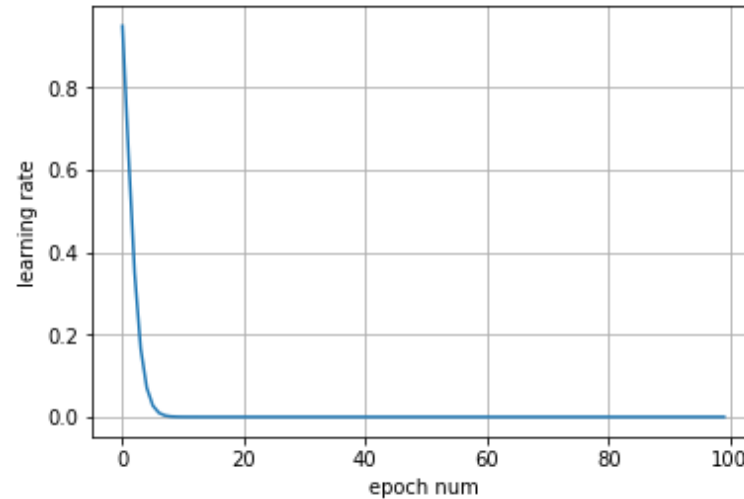
decay_rate(k) = 0.01



1/t decay

$$\alpha = \frac{k \times \alpha_0}{\sqrt{\text{epoch_num}}} = \frac{k \times \alpha_0}{\sqrt{t}}$$

$$\alpha_0 = 1$$
$$k = 0.95$$

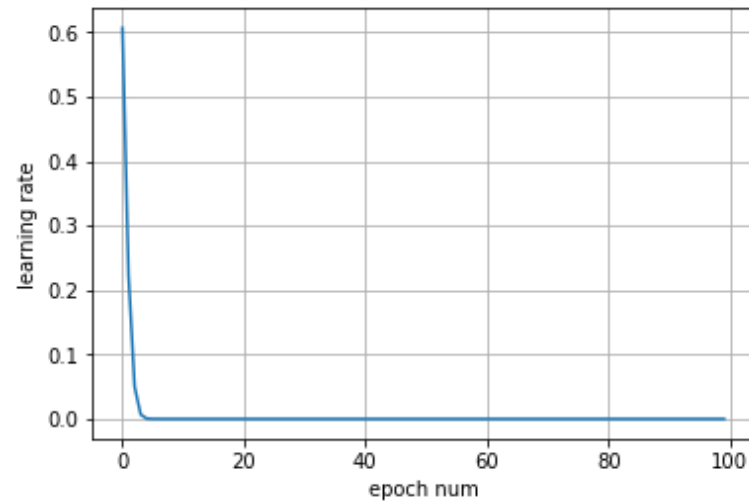


Exponential decay

$$\alpha = \alpha_0 \cdot \exp(-kt)$$

$$\alpha_0 = 1$$

$$k = 0.5$$

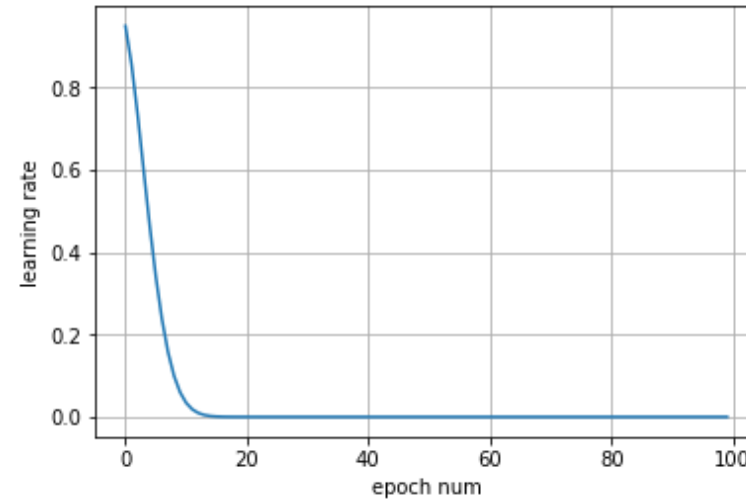


Exponential decay

$$\alpha = k^{epoch_num} \times \alpha_0 = k^t \times \alpha_0$$

$$\alpha_0 = 1$$

$$k = 0.95$$



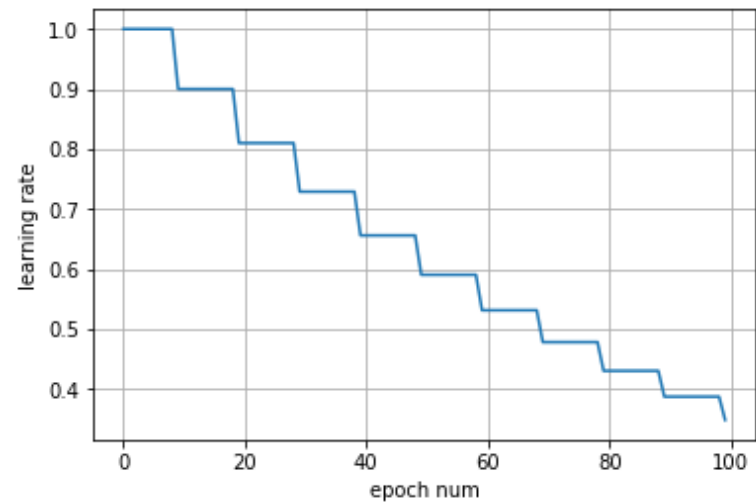
Discrete staircase

$\alpha = k \times \alpha$ with every few epochs

$$\alpha_0 = 1$$

$$k = 0.9$$

every 10 epochs



Learning rate decay

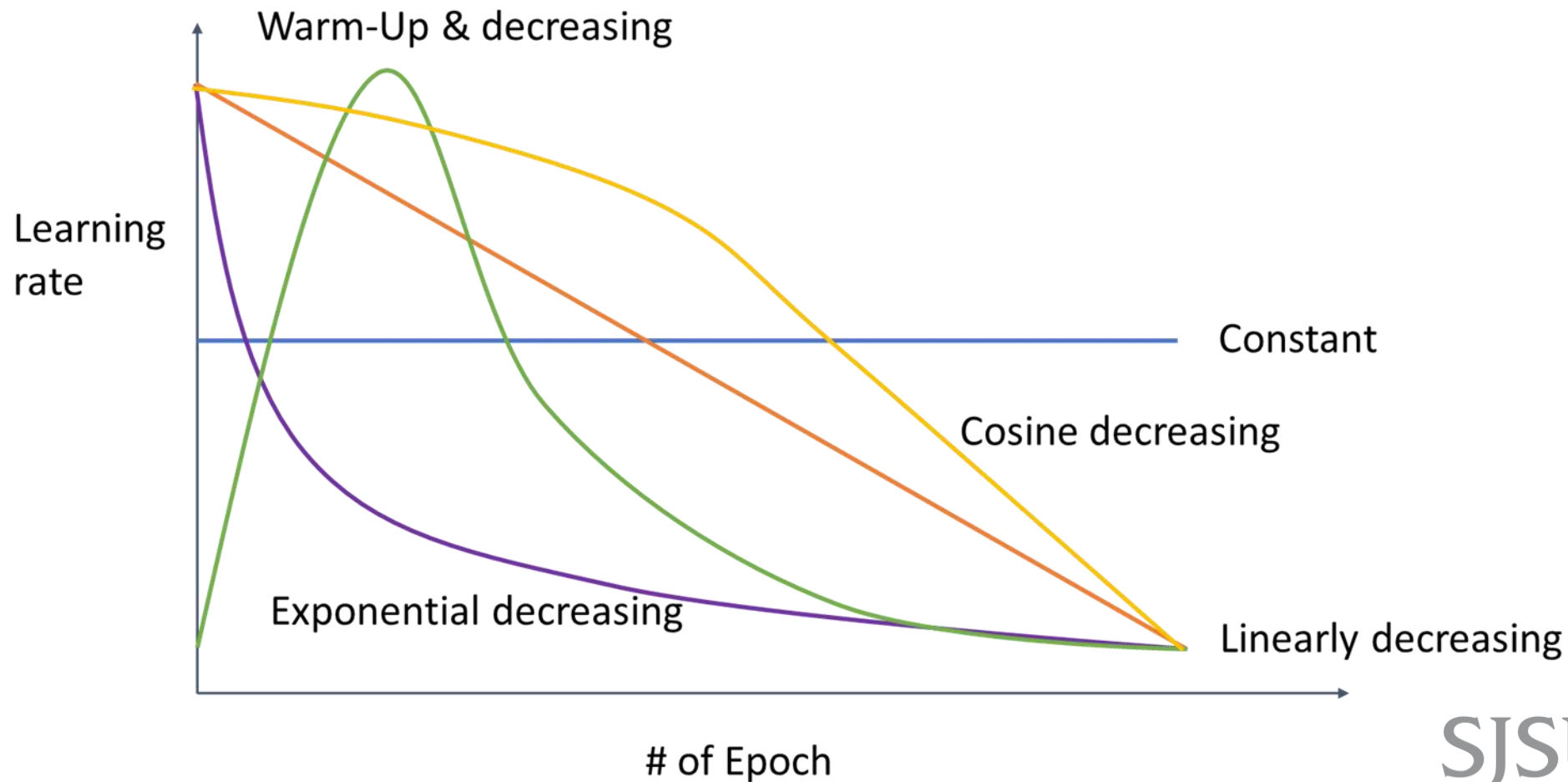
Slowing decreasing α

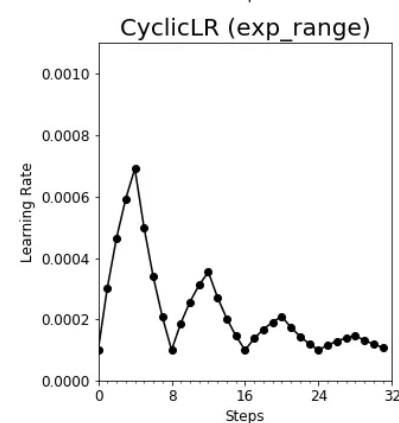
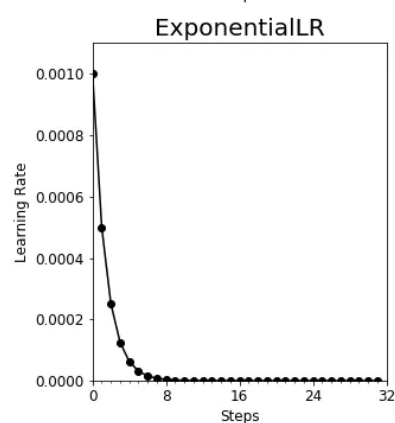
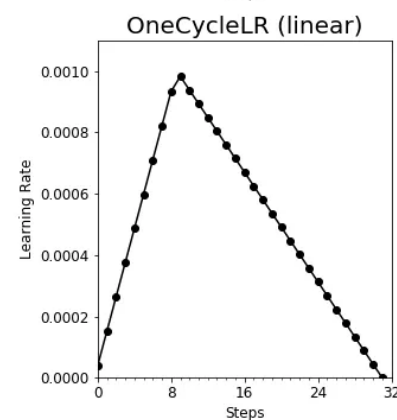
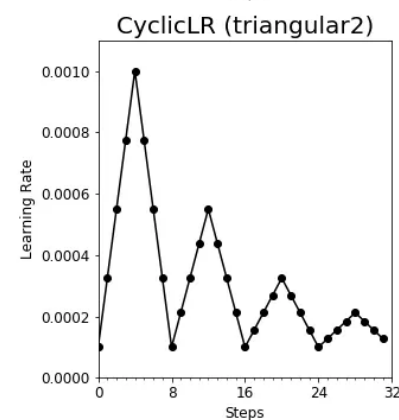
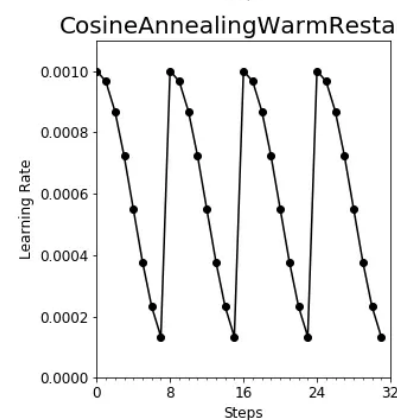
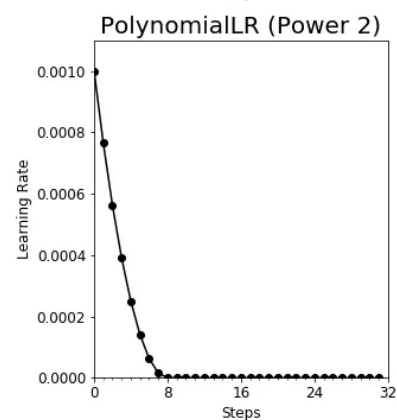
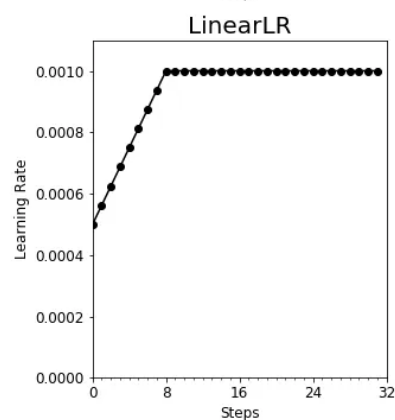
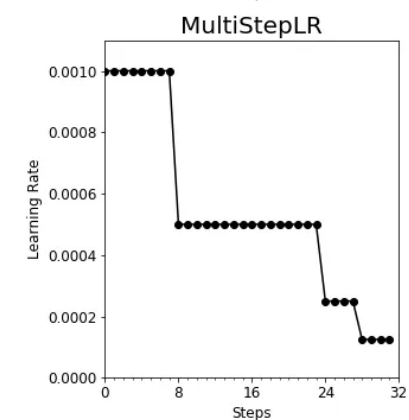
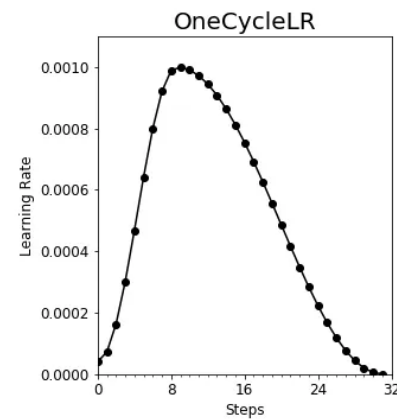
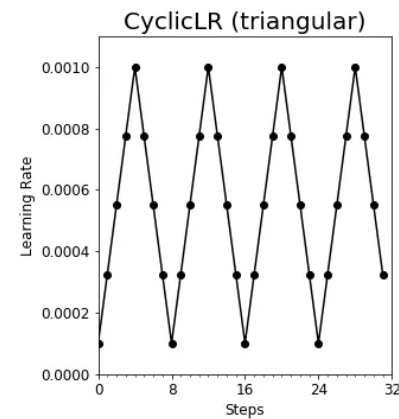
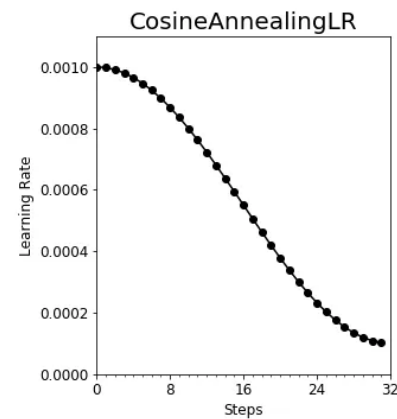
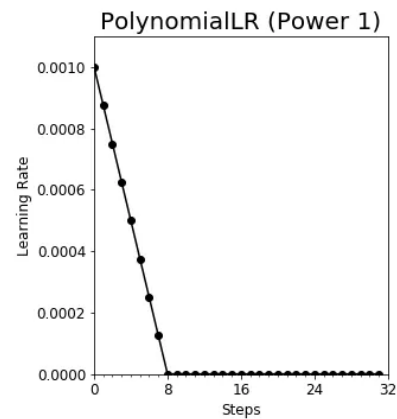
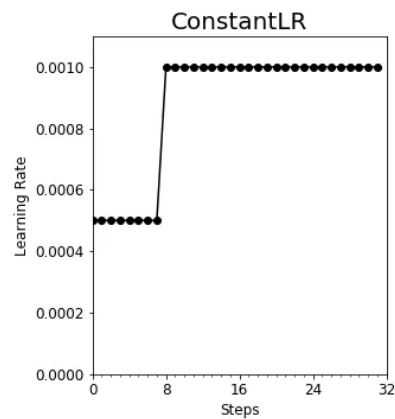
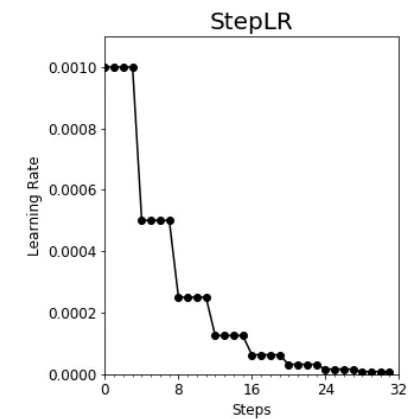
- 1/t decay: $\alpha = \frac{\alpha_0}{1 + kt}$ | $\alpha = \frac{k \times \alpha_0}{\sqrt{t}}$
- Exponentially decay $\alpha = \alpha_0 \cdot \exp(-kt)$ | $\alpha = k^t \times \alpha_0$
- Step decay: discrete staircase
 $\alpha = k \times \alpha$ with every t

Choosing the right step size

- Using a fixed step size
- Gradually shrinking the step size over time
- Warm-up & shrinking

Learning rate over Epoch

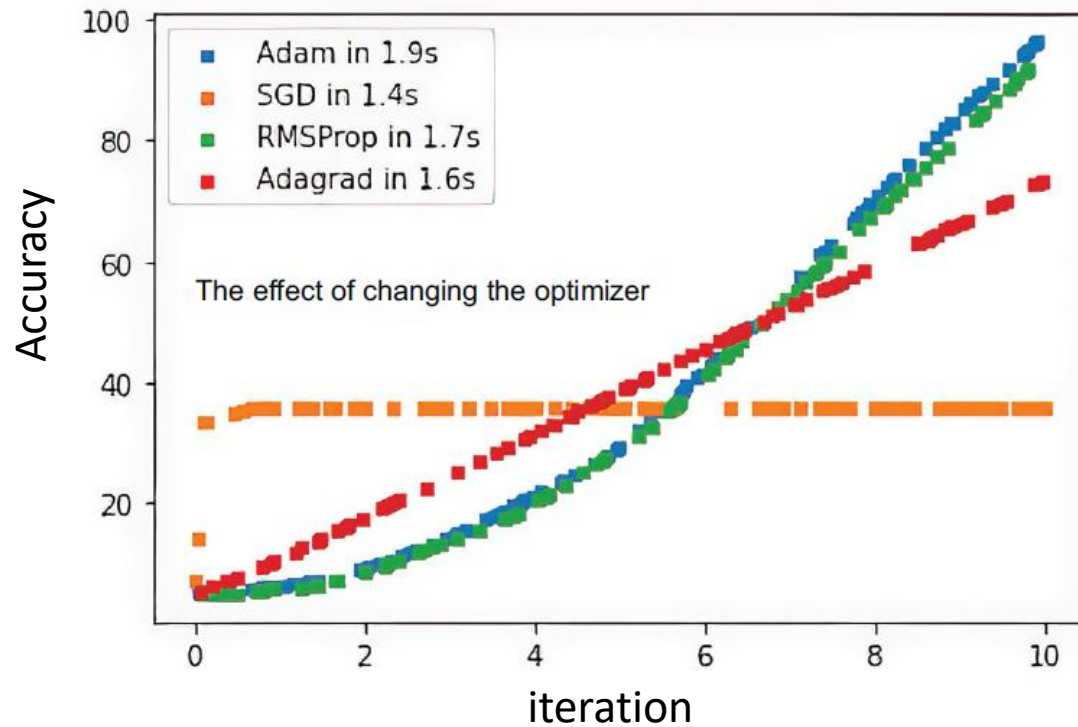




Parameter update (Faster optimization)

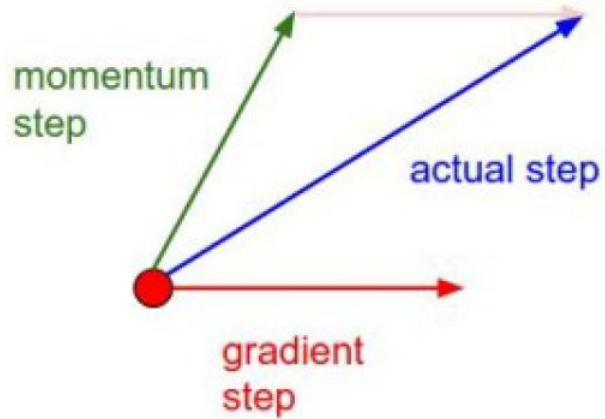
- Momentum-based Gradient optimizer
- Nesterov momentum → not cover
- Exponentially Weighted Averages
- Adagrad
- Rmsprop
- Adam
- Second order method: based on Newton's method, L-BFGS → not cover

Comparison of optimizer functions



Momentum update

Gradient descent update
 $x := x - \text{learning rate} * dx$



Momentum update

$$v = \beta * v - \text{learning rate} * dx$$

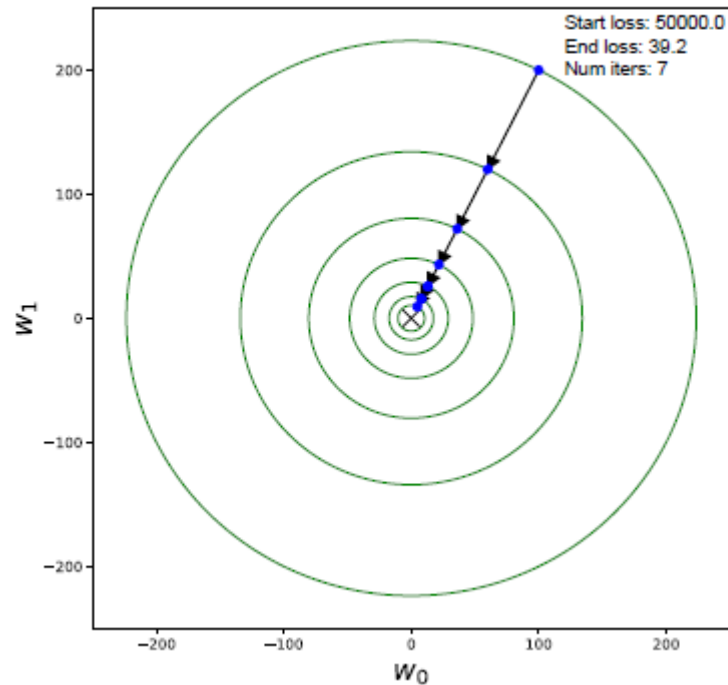
$$x := x + v$$

v : velocity

β : momentum

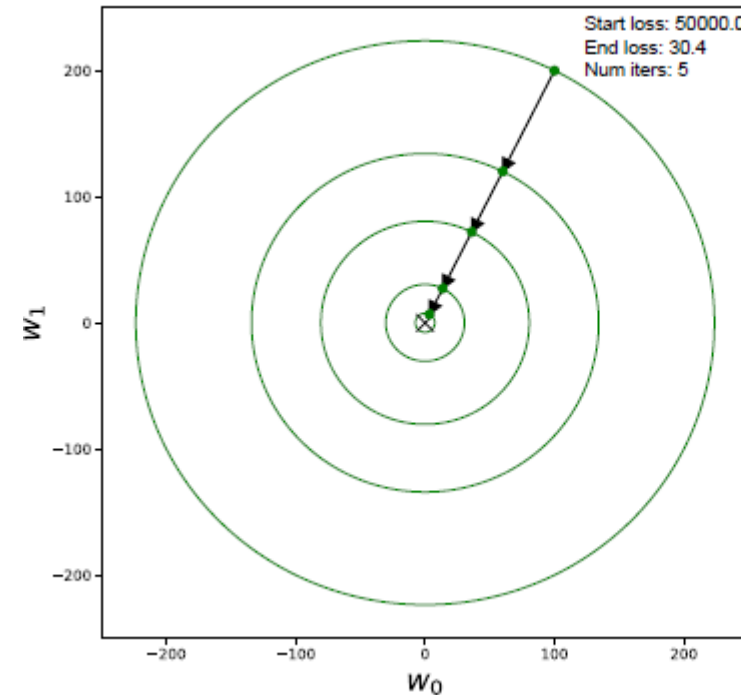
Gradient descent with/wo Momentum

Gradient descent without momentum for loss $L(w_0, w_1) = w_0^2 + w_1^2 = k$



(a) A trajectory through constant loss contours for gradient descent *without* momentum

Gradient descent with momentum for loss $L(w_0, w_1) = w_0^2 + w_1^2 = k$



(b) A trajectory through constant loss contours for gradient descent *with* momentum

Adagrad

Adaptive learning rate method

$$v := v + dx^{**2}$$

$$x := x - \text{learning_rate} * dx / (\text{sqrt}(v) + \text{eps})$$

v : sum of squared gradients

Normalized the parameter update step, element-wise.

Weights with high gradients will have effective learning rate reduces,

Weights with low gradients will have effective learning rate increases.

Gradient descent with momentum

$$V_{dW} = 0, V_{db} = 0$$

For every iteration t:

Compute V_{dW} , V_{db} using dW and db

$$V_{dW} := \beta_1 \cdot V_{dW} + (1 - \beta_1) \cdot dW$$

$$V_{db} := \beta_1 \cdot V_{db} + (1 - \beta_1) \cdot db$$

$$W := W - \alpha \cdot V_{dW}$$

$$b := b - \alpha \cdot V_{db}$$

RMS prop

Root Mean square prop

$$S_{dW} \neq 0, S_{db} \neq 0, \epsilon = 10^{-8}$$

For every iteration t:

Compute S_{dW} , S_{db} using dW and db

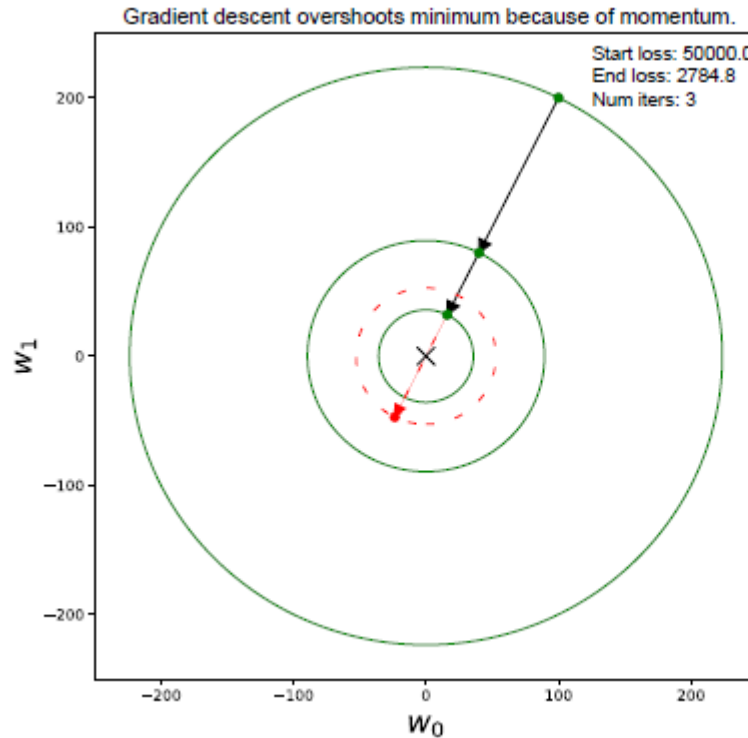
$$S_{dW} := \beta_2 \cdot S_{dW} + (1 - \beta_2) \cdot dW^2$$

$$S_{db} := \beta_2 \cdot S_{db} + (1 - \beta_2) \cdot db^2$$

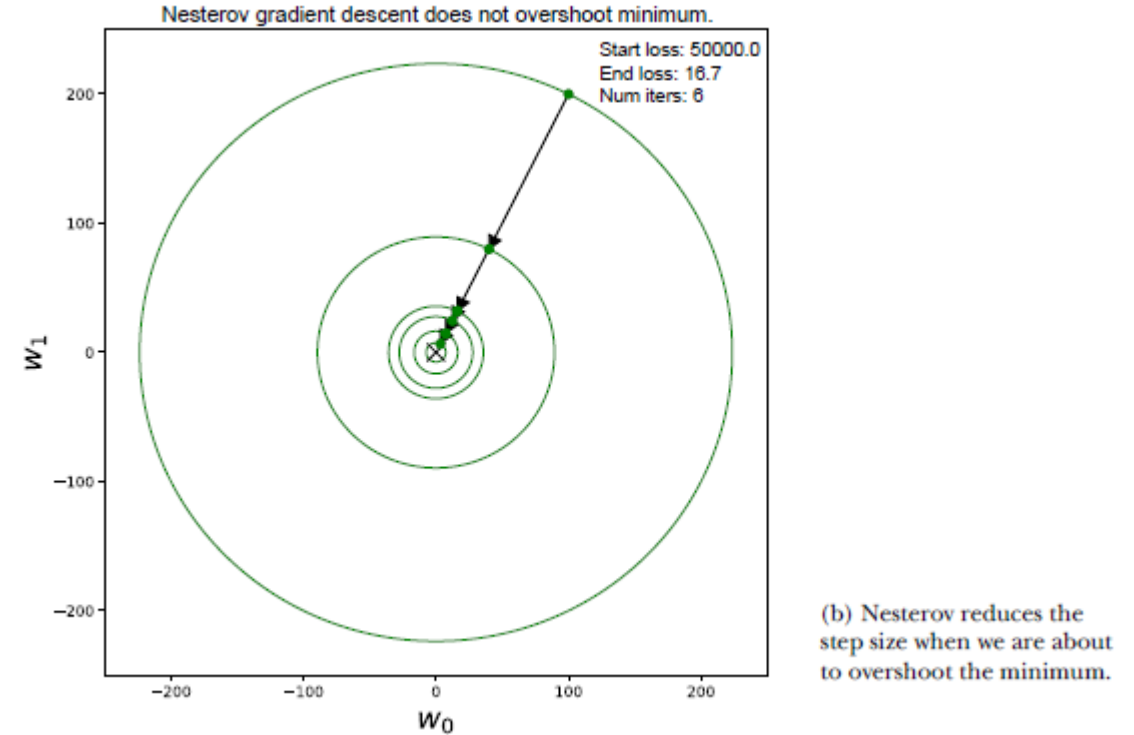
$$W := W - \alpha \cdot \frac{dW}{\sqrt{S_{dW} + \epsilon}}$$

$$b := b - \alpha \cdot \frac{db}{\sqrt{S_{db} + \epsilon}}$$

Nesterov accelerated gradient



(a) Momentum gradient descent overshooting the minimum. Loss decreases for a while and then increases (dotted circle and arrow).



(b) Nesterov reduces the step size when we are about to overshoot the minimum.

Adam Optimization

Adaption moment estimation

$$V_{dW} = 0, V_{db} = 0, S_{dW} = 0, S_{db} = 0, \epsilon = 10^{-8}$$

For every iteration t:

Compute V_{dW} , V_{db} , S_{dW} , S_{db} using dW and db

$$V_{dW} := \beta_1 \cdot V_{dW} + (1 - \beta_1) \cdot dW$$

$$V_{db} := \beta_1 \cdot V_{db} + (1 - \beta_1) \cdot db$$

$$S_{dW} := \beta_2 \cdot S_{dW} + (1 - \beta_2) \cdot dW^2$$

$$S_{db} := \beta_2 \cdot S_{db} + (1 - \beta_2) \cdot db^2$$

$$W := W - \alpha \cdot V_{dW}$$

$$b := b - \alpha \cdot V_{db}$$

Adam Optimization

Bias correction

$$V_{dW}^{cor} = \frac{V_{dW}}{1 - \beta_1^t}$$

$$V_{db}^{cor} = \frac{V_{db}}{1 - \beta_1^t}$$

$$S_{dW}^{cor} = \frac{S_{dW}}{1 - \beta_2^t}$$

$$S_{db}^{cor} = \frac{S_{db}}{1 - \beta_2^t}$$

$$W := W - \alpha \cdot \frac{V_{dW}^{cor}}{\sqrt{S_{dW}^{cor} + \epsilon}}$$

$$b := b - \alpha \cdot \frac{V_{db}^{cor}}{\sqrt{S_{db}^{cor} + \epsilon}}$$

Adam optimization

Hyper parameters choices

- α : need to be tune
- β_1 : 0.9 (momentum)
- β_2 : 0.999 (RMS prop)
- ϵ : 10^{-8}

Summary

SGD optimizer

$$W_j^t := W_j^{t-1} - \alpha \frac{\partial J}{\partial W}$$

Momentum optimizer

$$m^t = \beta \cdot m^{t-1} - \alpha \frac{\partial J}{\partial W}$$

$$W_j^t := W_j^{t-1} + m^t$$

$$W_j^t := W_j^{t-1} + \beta \cdot m^{t-1} - \alpha \frac{\partial J}{\partial W}$$

Weighted Averages

$$m^t = \beta \cdot m^{t-1} + (1 - \beta) \frac{\partial J}{\partial W}$$

$$W_j^t := W_j^{t-1} - \alpha \cdot m^t$$

$$W_j^t := W_j^{t-1} - \alpha \cdot [\beta \cdot m^{t-1} + (1 - \beta) \frac{\partial J}{\partial W}]$$

Adagrad

$$v^t = \sum_{i=0}^{t-1} \left(\frac{\partial J}{\partial W^i} \right)^2$$

$$W_j^t := W_j^{t-1} - \frac{\alpha}{\sqrt{v^t + \epsilon}} \cdot \frac{\partial J}{\partial W}$$

RMSprop

$$v^t = \beta \cdot v^{t-1} + (1 - \beta) \cdot \left(\frac{\partial J}{\partial W} \right)^2$$

$$W_j^t := W_j^{t-1} - \frac{\alpha}{\sqrt{v^t + \epsilon}} \cdot \frac{\partial J}{\partial W}$$

Adam

$$m^t := \beta_1 \cdot m^{t-1} + (1 - \beta_1) \cdot \frac{\partial J}{\partial W}$$

$$v^t = \beta_2 \cdot v^{t-1} + (1 - \beta_2) \cdot \left(\frac{\partial J}{\partial W} \right)^2$$

$$W_j^t := W_j^{t-1} - \alpha \cdot \frac{\widehat{m}^t}{\sqrt{\widehat{v}^t + \epsilon}} \cdot \frac{\partial J}{\partial W}$$

$$\widehat{m}^t = \frac{m^t}{1 - \{\beta_1\}^t}$$

$$\widehat{v}^t = \frac{v^t}{1 - \{\beta_2\}^t}$$

Today's Topics(2)

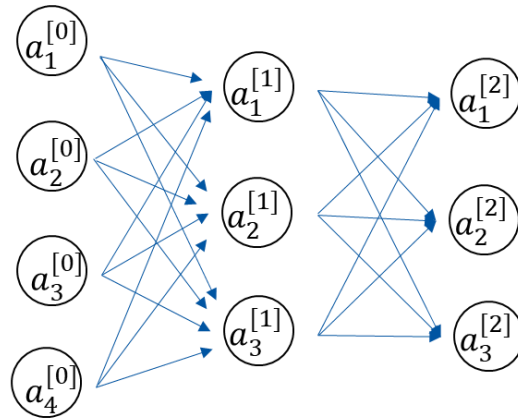
- Deep neural network (DNN) vs. Convolutional neural network (CNN)
- Convolution layer
- Convolution (CONV)

Deep neural network (DNN)

; Artificial neural network (ANN)

It can be applied to regression or classification.

- Regression: predict continuous valued output
- Classification: predict categorical value

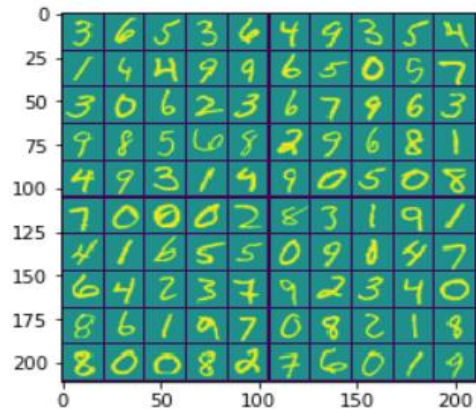


Input layer

Hidden layer

Output layer

Digital representation of an image



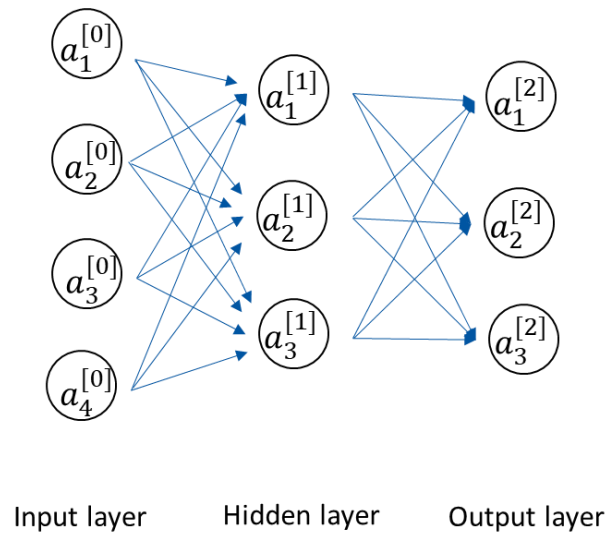
	0	1	2	3	4	5	6	7	8	9	...	391	392	393	394	395	396	397	398	399	y
0	0	0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0	5
1	0	0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0	9
2	0	0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0	7
3	0	0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0	6
4	0	0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0	5

- Grayscale image is a matrix of pixels (picture elements)
- Dimensions of this matrix are called image resolution (e.g. 28 x 28)
- Each pixel stores its brightness (or intensity) ranging from 0 to 255, 0 intensity corresponds to black color
- Color images store pixel intensities for 3 channels: red, green, blue

Why not Deep Neural Network?



$1000 \times 1000 \times 3 = 3\text{M}$



Why not Deep Neural Network?

- Why not simply use a regular deep neural network with fully connected layers for image recognition tasks?
- Unfortunately, although this works fine for small images (e.g., MNIST), it breaks down for larger images because of the huge number of parameters it requires.
- For example, a 100×100 image has 10,000 pixels, and if the first layer has just 1,000 neurons (which already severely restricts the amount of information transmitted to the next layer),
- this means a total of 10 million connections. And that's just the first layer.
- CNNs solve this problem using [partially connected layers](#).

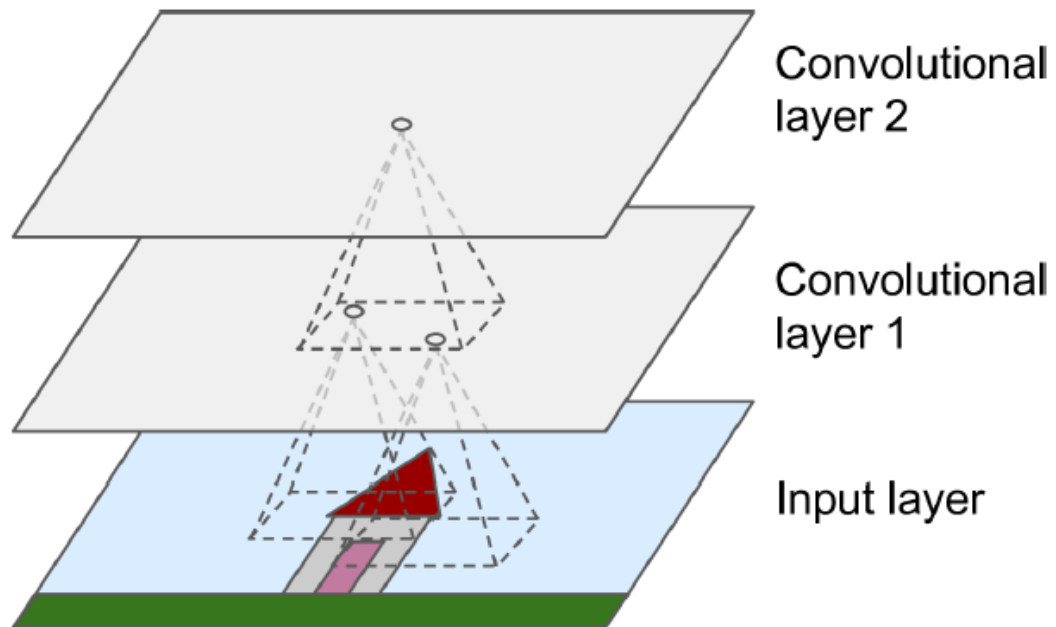
Convolutional Layer

- The most important building block of a CNN is the *convolutional layer*:
- Neurons in the first convolutional layer are not connected to every single pixel in the input image, but only to pixels in their receptive fields.
- Neuron in the second convolutional layer is connected only to neurons located within a small rectangle in the first layer.
- This architecture allows the network to concentrate on low-level features in the first hidden layer, then assemble them into higher-level features in the next hidden layer, and so on.
- This hierarchical structure is common in real-world images, which is one of the reasons why CNNs work so well for image recognition.

Advantages of CNN over DNN for image classification

- CNN has **fewer parameters** than a fully connected DNN
- CNN can detect a particular **feature anywhere** on the image instead of detecting only in that **particular location**.
- CNN's architecture is able to identify pixels' organization (how each pixel is located).

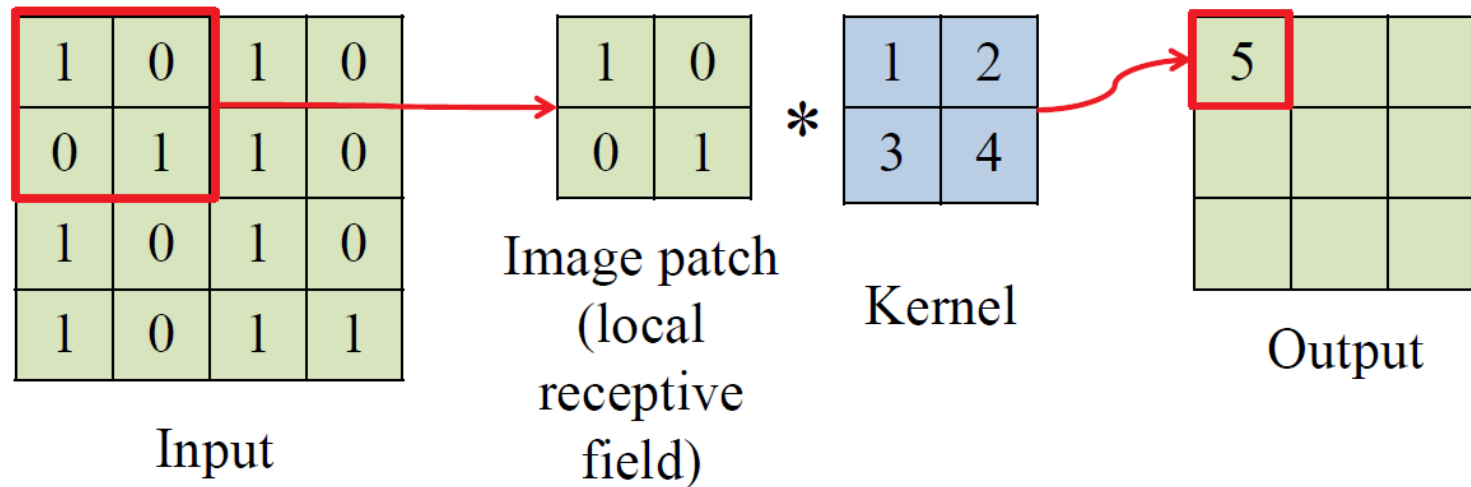
Convolutional layer



Source: *Hands-On Machine Learning with Scikit-Learn & TensorFlow*

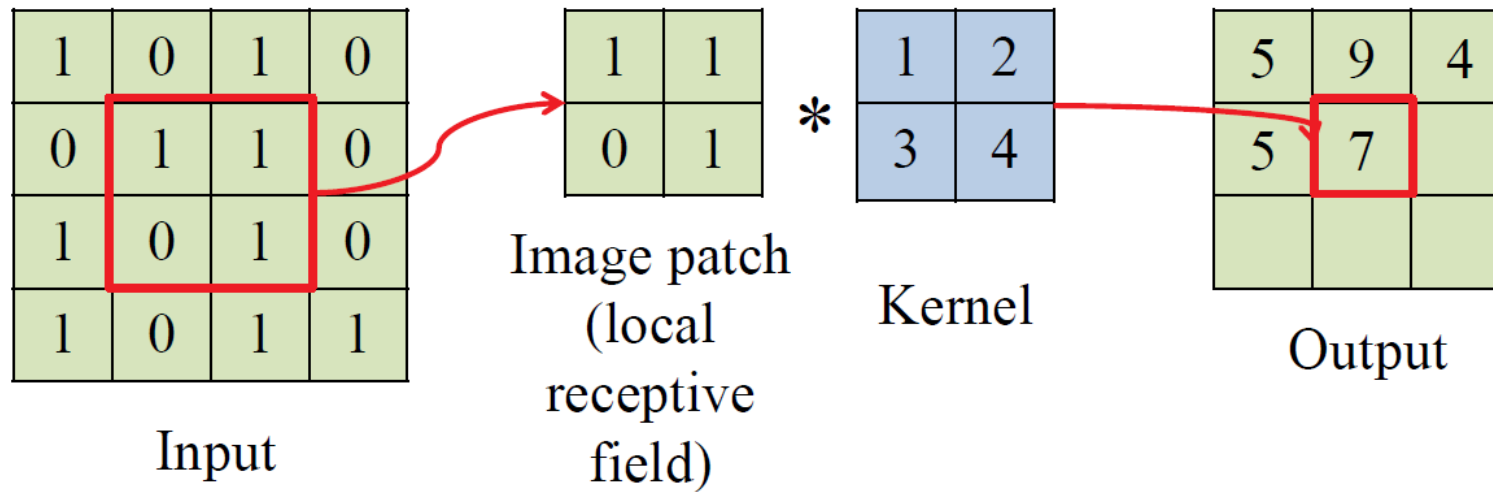
Convolutions

Convolution is a dot product of a **kernel** (or filter) and a patch of an image (**local receptive field**) of the same size

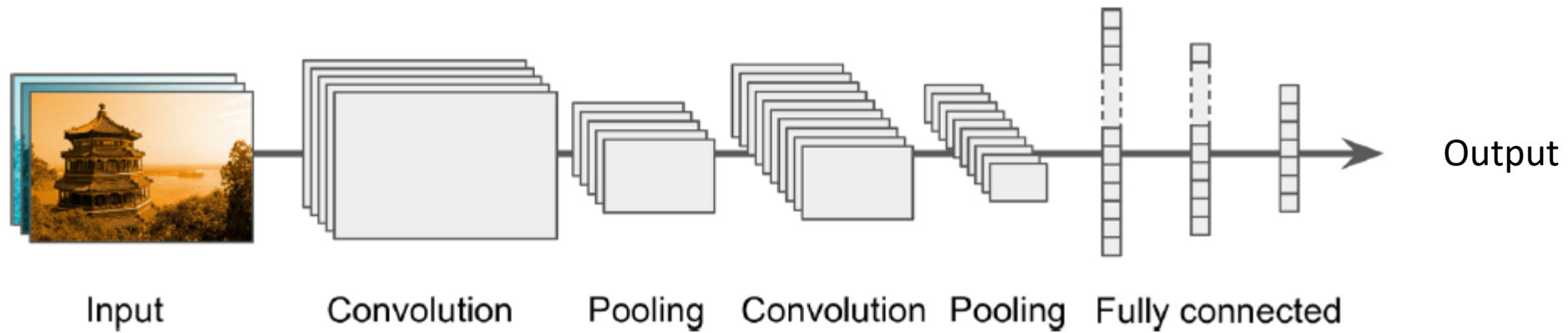


Convolutions

Convolution is a dot product of a **kernel** (or filter) and a patch of an image (**local receptive field**) of the same size



CNN Architecture



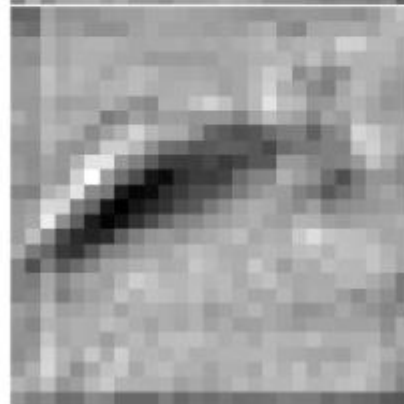
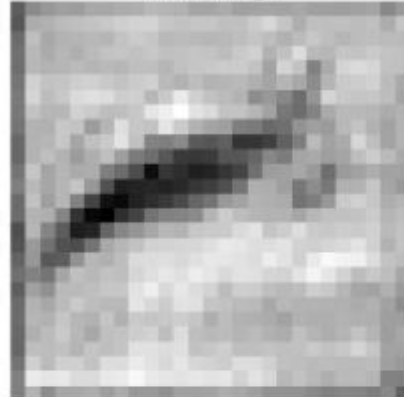
Output of Convolutional layers



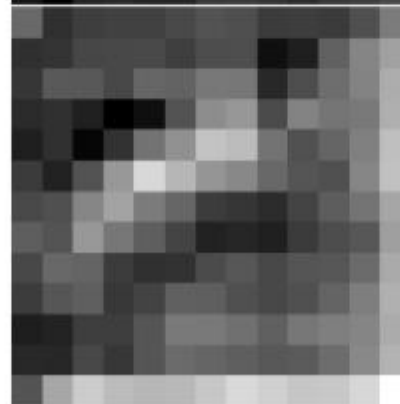
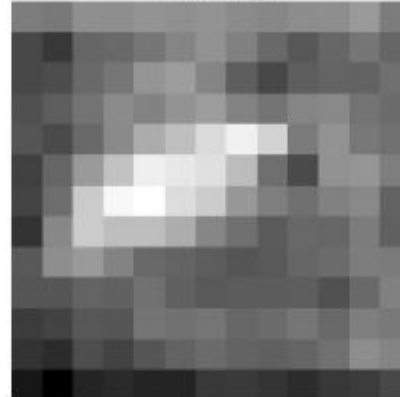
convolution_1



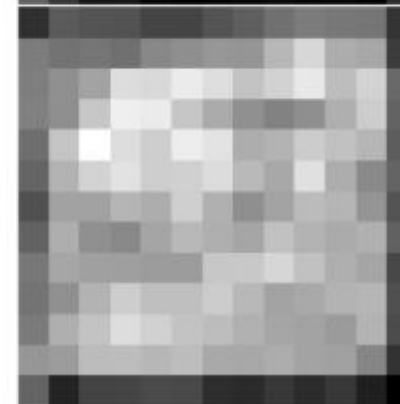
convolution_2



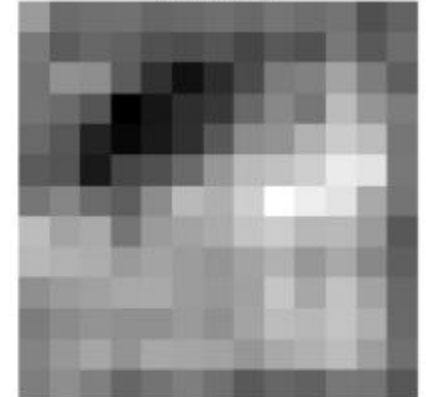
convolution_3



convolution_4



convolution_5



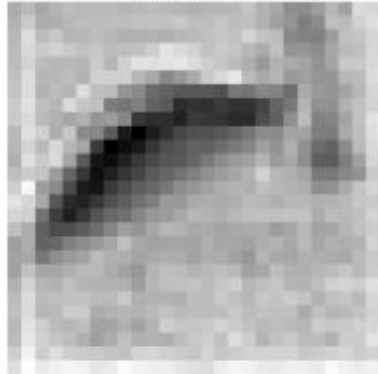
Output of Convolutional layers



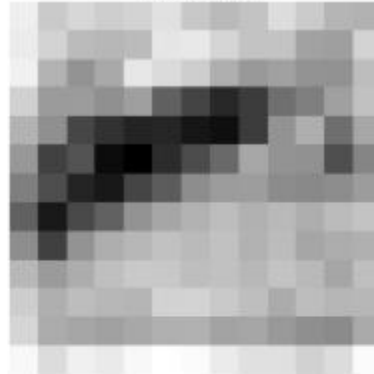
Conv1
[55×55×96]



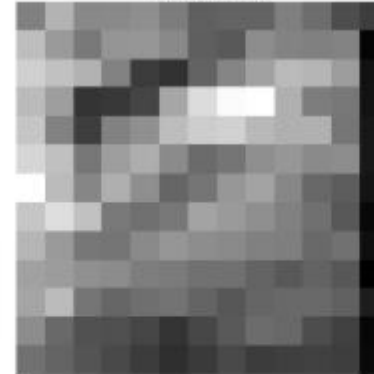
Conv2
[27×27×256]



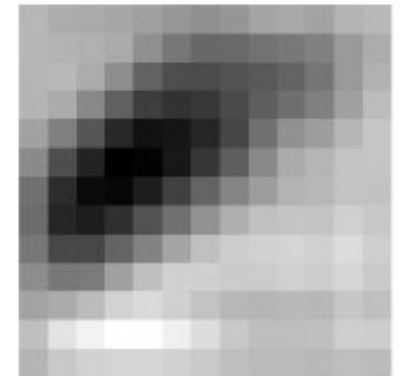
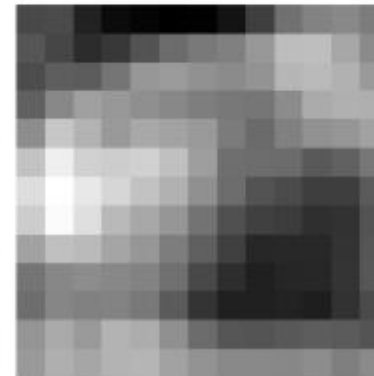
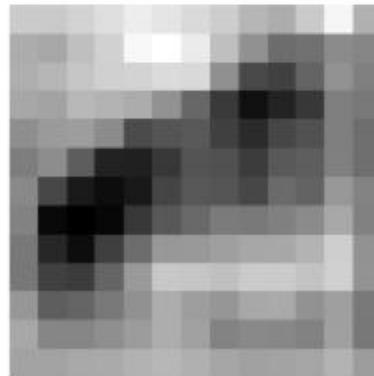
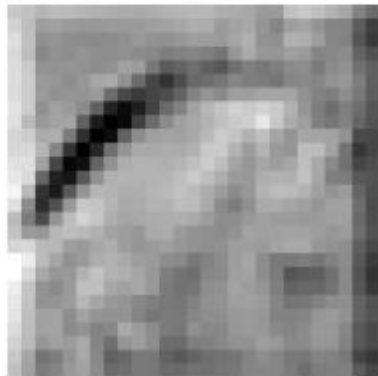
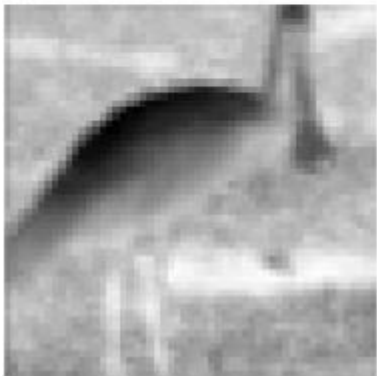
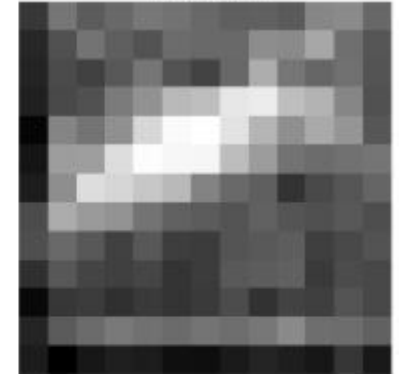
Conv3
[13×13×384]



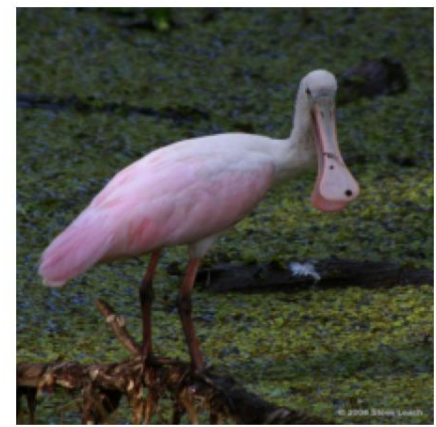
Conv4
[13×13×384]



Conv5
[13×13×256]



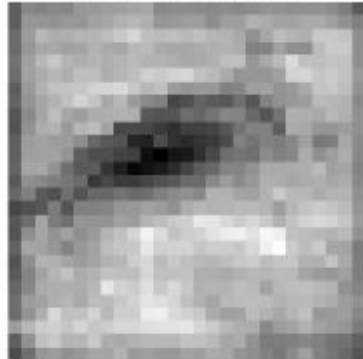
Output of Convolutional layers



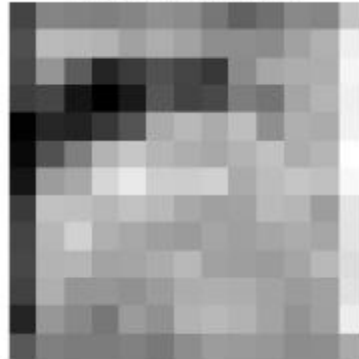
Conv1 [55×55] ch0



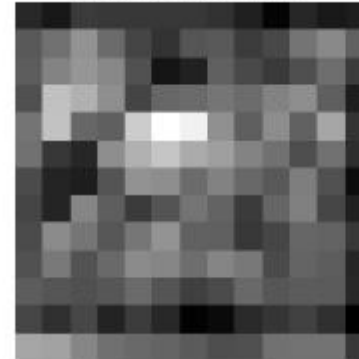
Conv2 [27×27] ch0



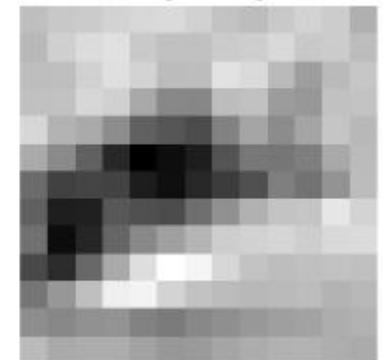
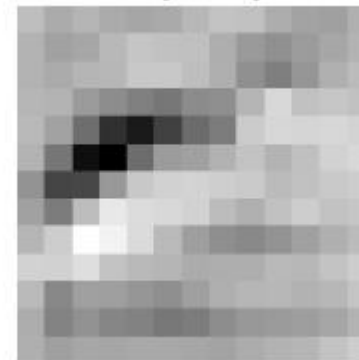
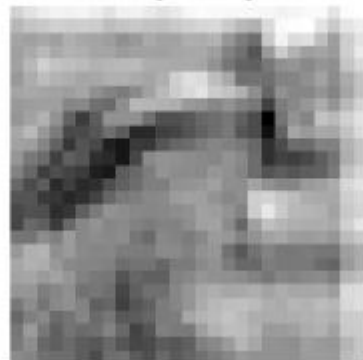
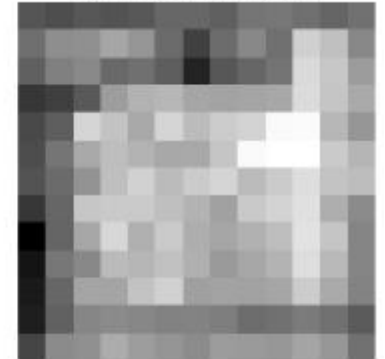
Conv3 [13×13] ch0



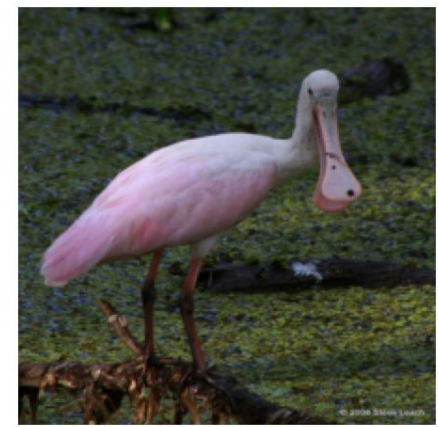
Conv4 [13×13] ch0



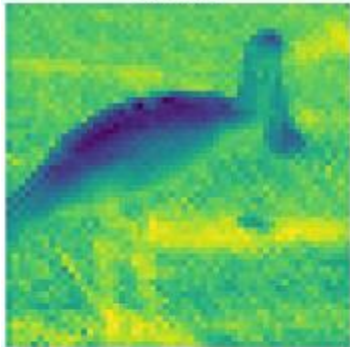
Conv5 [13×13] ch0



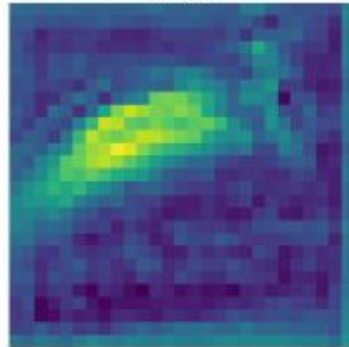
Output of Convolutional layers



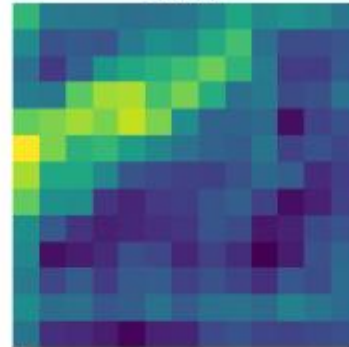
CONV1 Random
[55,55]



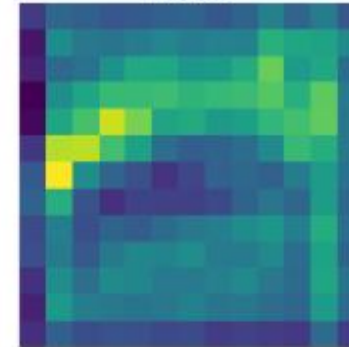
CONV2 Random
[27,27]



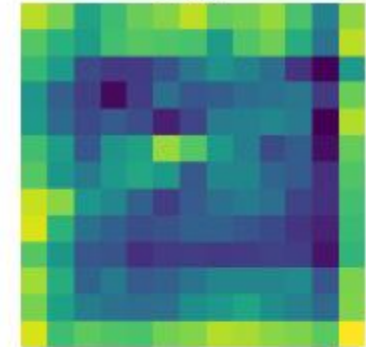
CONV3 Random
[13,13]



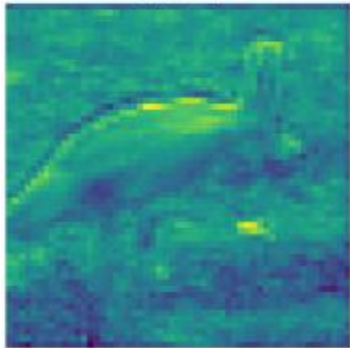
CONV4 Random
[13,13]



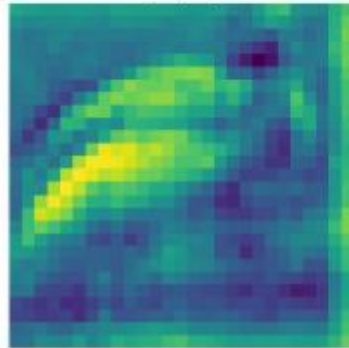
CONV5 Random
[13,13]



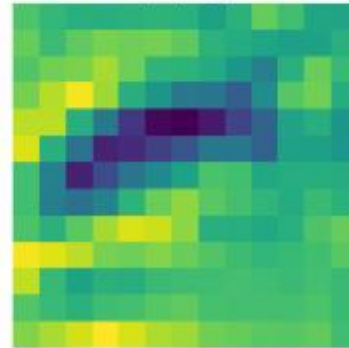
CONV1 Pretrained
[55,55]



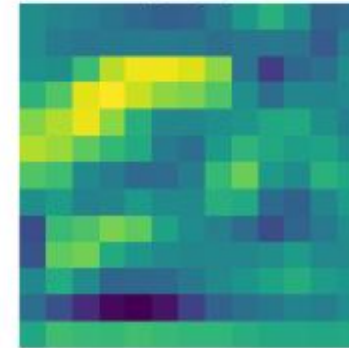
CONV2 Pretrained
[27,27]



CONV3 Pretrained
[13,13]



CONV4 Pretrained
[13,13]



CONV5 Pretrained
[13,13]

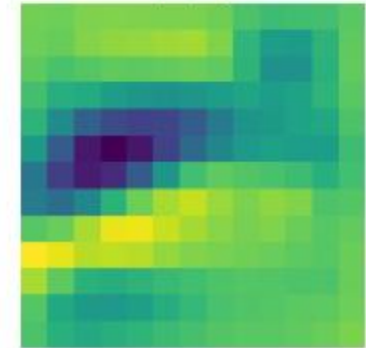
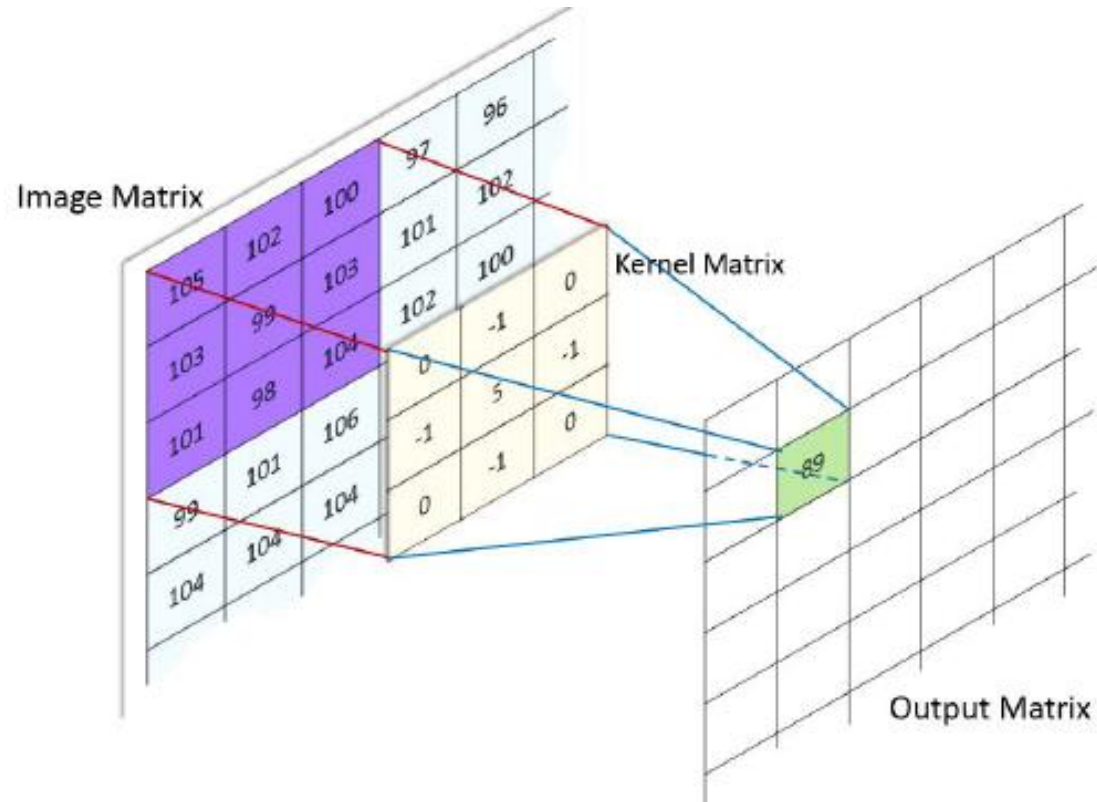
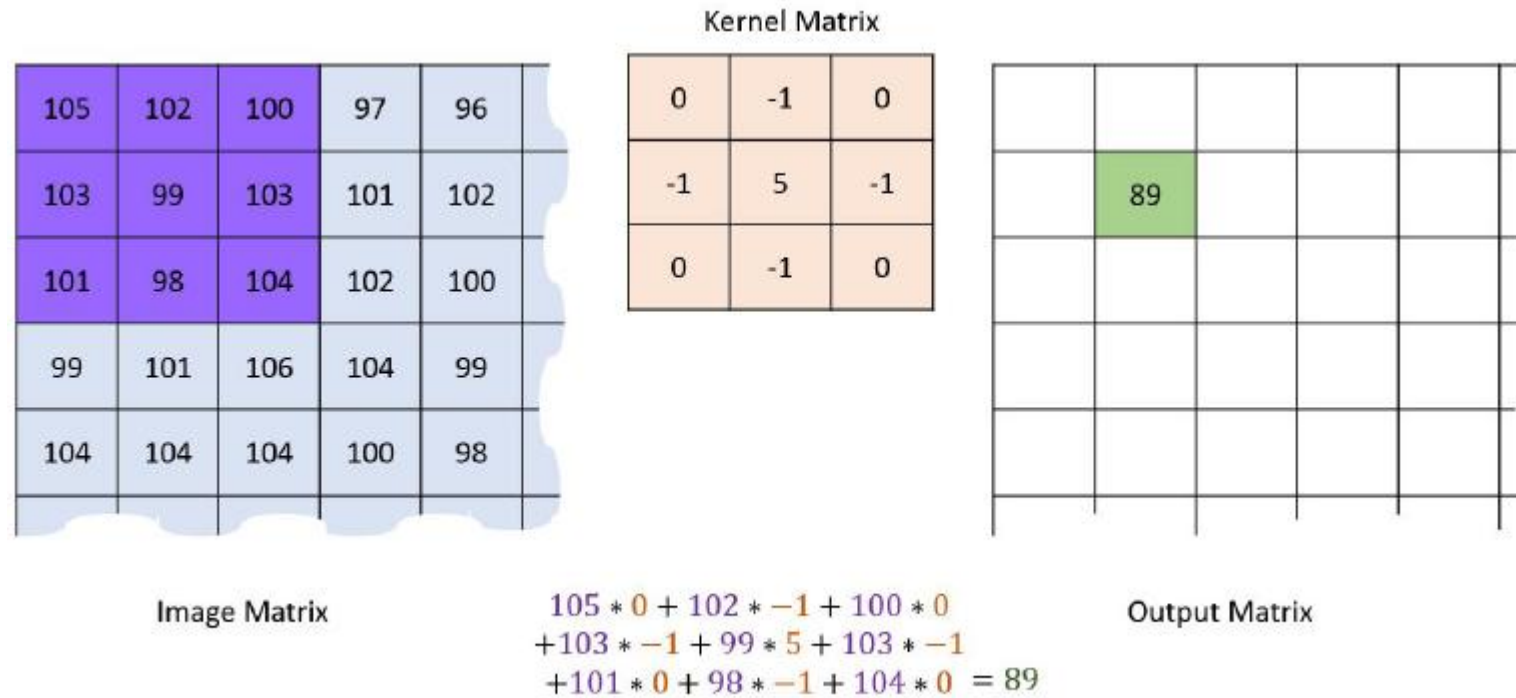


Image convolution operation



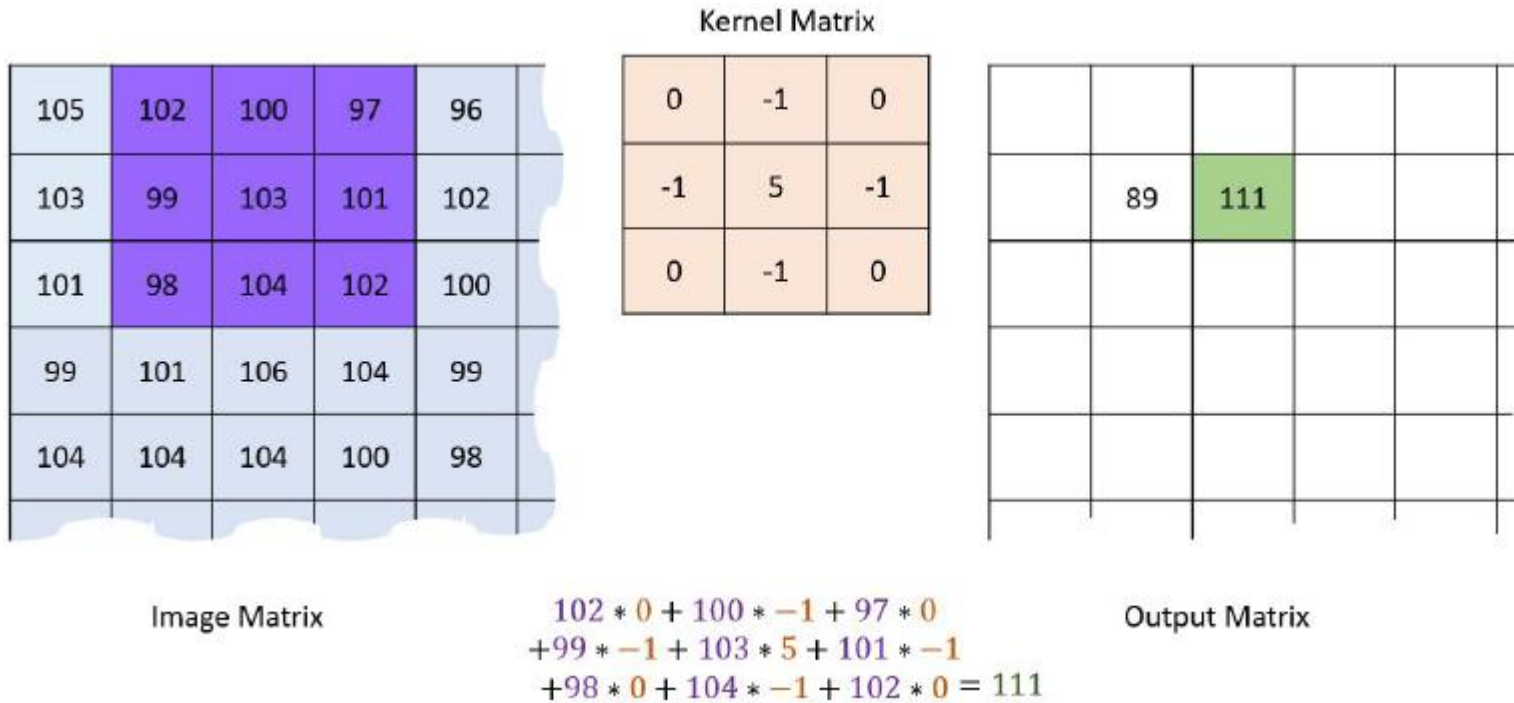
Source: Machinelearningguru.com/computer_vision/basics/convolution/image_convolution_1.html

Image convolution operation



Source: [Machinelearningguru.com/computer_vision/basics/convolution/image_convolution_1.html](https://machinelearningguru.com/computer_vision/basics/convolution/image_convolution_1.html)

Image convolution operation



Source: [Machinelearningguru.com/computer_vision/basics/convolution/image_convolution_1.html](https://machinelearningguru.com/computer_vision/basics/convolution/image_convolution_1.html)

Convolution operation before activation

X_1	X_2	X_3	X_4
X_5	X_6	X_7	X_8
X_9	X_{10}	X_{11}	X_{12}
X_{13}	X_{14}	X_{15}	X_{16}

 *

W_1	W_2	W_3
W_4	W_5	W_6
W_7	W_8	W_9

 =

Z_1	Z_2
Z_3	Z_4

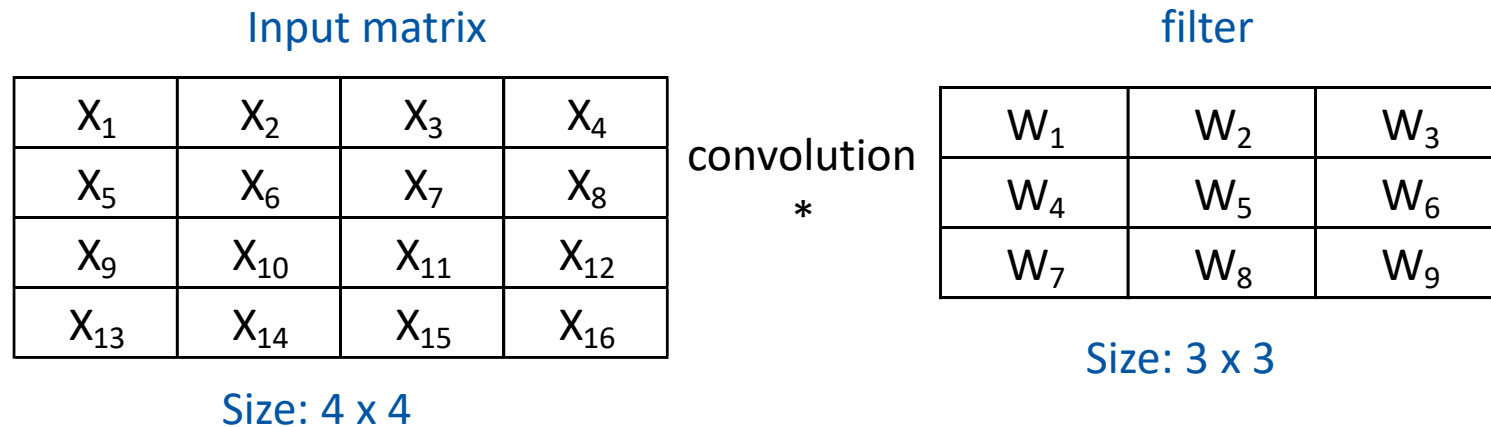
$$\begin{aligned} Z_1 = & X_1 \times W_1 + X_2 \times W_2 + X_3 \times W_3 \\ & + X_5 \times W_4 + X_6 \times W_5 + X_7 \times W_6 \\ & + X_9 \times W_7 + X_{10} \times W_8 + X_{11} \times W_9 \end{aligned}$$

$$\begin{aligned} Z_3 = & X_5 \times W_1 + X_6 \times W_2 + X_7 \times W_3 \\ & + X_9 \times W_4 + X_{10} \times W_5 + X_{11} \times W_6 \\ & + X_{13} \times W_7 + X_{14} \times W_8 + X_{15} \times W_9 \end{aligned}$$

$$\begin{aligned} Z_2 = & X_2 \times W_1 + X_3 \times W_2 + X_4 \times W_3 \\ & + X_6 \times W_4 + X_7 \times W_5 + X_8 \times W_6 \\ & + X_{10} \times W_7 + X_{11} \times W_8 + X_{12} \times W_9 \end{aligned}$$

$$\begin{aligned} Z_4 = & X_6 \times W_1 + X_7 \times W_2 + X_8 \times W_3 \\ & + X_{10} \times W_4 + X_{11} \times W_5 + X_{12} \times W_6 \\ & + X_{14} \times W_7 + X_{15} \times W_8 + X_{16} \times W_9 \end{aligned}$$

Convolution operation in activation



g : activation function
 b : bias

$$= \begin{bmatrix} g(Z_1 + b) & g(Z_2 + b) \\ g(Z_3 + b) & g(Z_4 + b) \end{bmatrix} = \begin{bmatrix} a_1 & a_2 \\ a_3 & a_4 \end{bmatrix}$$

Size: 2 x 2